



Go-программист разработка программ, ботов и веб-сервисов

Модуль 1 - Модуль 1. Погружение в Go и IT

Урок 1 - Добро пожаловать! Go — язык будущего от Google

Шаг 02:

Миссия 1 — “Секретная комната создателей

Ты попал в архив Google, где хранятся имена трёх легендарных разработчиков. Но охранник пустит тебя внутрь только если ты угадаешь всех правильно.

Вопрос: Кто был среди создателей Go? (выбери все верные) а) Роб Пайк b) Кен Томпсон c) Марк Цукерберг d) Роберт Грисмер  Навык: История и личности Go

Ответ:

- **Роб Пайк**
- **Кен Томпсон**
- Марк Цукерберг
- **Роберт Грисмер**

Шаг 03:

В серверной комнате стоит компьютер с Go-кодом. Чтобы попасть дальше, нужно догадаться, что выведет программа:

```
package main
import "fmt"
func main() {
```

```
    fmt.Println("Привет, мир!")  
}
```

Ответ:

- Ошибка
- **Привет, мир!**
- Go forever!

Шаг 04:

Миссия 3 — “Турбо-гонка”

В Google тестируют новый язык. Нужно понять, почему он такой быстрый.

Вопрос: Что делает Go особенно подходящим для огромных проектов?

Ответ:

- Он очень медленный, но надёжный
- **Он быстрый, простой и умеет выполнять много задач одновременно**
- Он использует только один поток работы

Шаг 05:

Миссия 4 — “Параллельная вселенная”

У тебя есть программа, которая одновременно скачивает файл и обрабатывает запросы пользователей. Как называется «суперспособность» Go, которая это позволяет?

a) goroutines b) multitasking++ c) hyperthreads

 Навык: Понимание параллельности в Go

Ответ:

- **goroutines**
- multitasking++
- hyperthreads

Шаг 06:

Миссия 5 — “Секрет Docker”

На стене висит плакат с надписью «Docker». Твоё задание — вспомнить, на каком языке он написан.

 Навык: Знание применения Go в реальных проектах

Ответ:

- Python
- **Go**
- Java

Урок 2 - Где используется Go сайты, игры, боты, сервисы

Шаг 02:

Миссия 1 — “Бонус: Шифр будущего”

Расшифруй фразу, если каждое слово заменено на его первую букву: Б, И, П, П, В, М, П
(Подсказка: это причины учить Go, из текста урока)

(Правильный ответ: Быстро, Интересно, Простой, Полезный, Востребованный, Масштабируемый, Перспективный)

🔗 Навык: Закрепление преимуществ Go

Ответ:

- **Быстро, Интересно, Простой, Полезный, Востребованный, Масштабируемый, Перспективный**
- Быстро, Простой, Полезный, Востребованный, Масштабируемый, Перспективный
- Быстро, Интересно, Простой, Полезный, Востребованный, Масштабируемый

🔗 **Шаг 03:**

Миссия 1 — Суперсайт

Представь, что твой сайт посещают тысячи людей одновременно. Go поможет, чтобы он не зависал .

Вопрос: Что можно сделать с помощью Go? а) Сделать быстрый сайт б) Нарисовать картину с) Погладить кота

Ответ:

- **Сделать быстрый сайт**
- Нарисовать картину
- Погладить кота

🔗 **Шаг 04:**

Миссия 3 — Игра без лаго

Ты сделал онлайн-игру, и в неё зашли все твои друзья. Вопрос: Почему Go подходит для игр?

Ответ:

- **Он умеет быстро обрабатывать события**
- Он умеет играть вместо тебя
- Он рисует 3D-модели

🔗 **Шаг 05:**

Миссия 4 — Бот-помощни

Твой бот в чате отвечает на вопросы быстрее, чем одноклассник поднимает руку.

Вопрос: Что умеет бот на Go?

Ответ:

- **Работать 24 часа в сутки**
- Спать на перемене
- Играть в футбол

🔗 **Шаг 06:**

Миссия 5 — Облако данны

Ты сделал сервис, куда друзья могут загружать фото и документы.

Вопрос: Для чего Go используют в облачных сервисах? а) Чтобы они работали быстро и не ломались б) Чтобы рисовать облака на картинке с) Чтобы хранить игрушки

Ответ:

- **Чтобы они работали быстро и не ломались**
- Чтобы рисовать облака на картинке
- Чтобы хранить игрушки

Урок 3 - Как школьники зарабатывают на Go реальные истории и план

Шаг 02:

Миссия 1 — “Бот, который всё помнит

Артём сделал бота для школы, и потом его заказала фирма.

Вопрос: С чего он начал свой путь?

Ответ:

- С создания большой социальной сети
- **С полезного проекта для своей школы**
- С игры в шахматы

Шаг 03:

Миссия 2 — “Сервер без лагов

Марк сделал игровой сервер и продал его стримерам. Вопрос: Что помогло Марку получить первый заказ?

Ответ:

- **Он сделал сервер, который работал быстро и без лагов**
- Он продал сервер в магазине игрушек
- Он купил сервер у друга

Шаг 04:

Миссия 3 — “Сервис для фрилансеров

Соня сделала сервис с платной подпиской.

Вопрос: Сколько подписчиков у Сони было через 4 месяца?

Ответ:

- 12
- **120**
- 1200

Шаг 05:

Миссия 4 — “Мини-облако

Илья сделал облачное хранилище для фото.

Вопрос: Кто стал его первым бизнес-клиентом?

Ответ:

- **Фотостудия**
- Учитель информатики
- Друг по школе

Шаг 06:

Миссия 5 — “Помощник стримера

Катя написала программу, которая показывала донаты и благодарила доноров.

Вопрос: Сколько клиентов-стримеров у неё стало через пару недель?

Ответ:

- 2
- 5
- 50

Урок 4 - Как запускать код Go онлайн 3 быстрых способа

Урок 5 - Первая программа на Go от установки до запуска

Шаг 02:

Миссия 2 — “Тайный свиток кода

В VS Code ты создаёшь файл, в который поместишь свои первые строки на Go.

Вопрос: Какое расширение должен иметь файл с кодом Go?

Ответ:

- **.go**
- .golang
- .txt

Шаг 03:

Миссия 3 — “Заклинание запуска

Ты пишешь волшебное заклинание, чтобы на экране появилось послание миру.

Вопрос: Какая команда на Go выводит текст на экран?

Ответ:

- `fmt.Write()`
- **`fmt.Println()`**
- `fmt.Text()`

Шаг 04:

Миссия 5 — “Изменение заклинания

Теперь ты волшебник, который умеет менять реальность (ну, хотя бы надписи).

Вопрос: Что будет, если изменить текст в кавычках на "Привет, школа!" ?

Ответ:

- Ошибка
- **Программа выведет новый текст**
- Компьютер зависнет

Урок 6 - Как запускать программы на Go на Stepik

Урок 7 - Установка и настройка Go за 10 минут

Шаг 02:

Миссия 1 — “Скачай артефакт

Ты нашёл секретный сайт разработчиков Google (<https://go.dev/dl/>). На нём лежат файлы с Go. Нужно выбрать правильный, чтобы компьютер стал «умнее».

Вопрос: Какой файл возьмёшь, если у тебя Windows?

Ответ:

- go1.xx.pkg
- **go1.xx.msi**
- go1.xx.tar.gz

💡 **Шаг 03:**

Миссия 2 — “Проверка портала

Go установлен, но нужно убедиться, что он «откликается».

Вопрос: Какую команду нужно ввести, чтобы проверить версию Go?

Ответ:

- go test
- **go version**
- go check

💡 **Шаг 04:**

Миссия 3 — “Первое заклинание

Ты готов написать свою первую магическую фразу на Go.

Вопрос: Какая команда запустит файл main.go?

Ответ:

- go start main.go
- **go run main.go**
- run go main.go

💡 **Шаг 05:**

Миссия 4 — “Секретное оружие

Чтобы писать код быстрее, нужны специальные инструменты.

Вопрос: Какой редактор для GO?

Ответ:

- **VS Code**
- Word
- Paint

📖 **Урок 8 - Как читать документацию по Go и находить ответы**

📖 **Урок 9 - Викторина что ты знаешь о Go и IT**

💡 **Шаг 02:**

В каком году был впервые выпущен язык Go?

Ответ:

- 2000
- **2009**
- 2015
- 2020

🔗 Шаг 03:

Что означает выражение «Hello, World!» в программировании?

Ответ:

- Название популярной игры
- **Первая программа, которая выводит приветствие на экран**
- Ошибка компиляции
- Новая версия языка Go

🔗 Шаг 04:

Для чего чаще всего используется язык Go?

Ответ:

- Создание мобильных игр
- **Разработка веб-сайтов, серверов и ботов**
- Обработка текстов и документов
- Создание музыкальных приложений

🔗 Шаг 05:

Что такое goroutines в языке Go?

Ответ:

- Тип данных
- **Функции, работающие параллельно (конкурентность)**
- Встроенный веб-сервер
- Ошибка компиляции

🔗 Шаг 06:

Почему Go называют языком будущего?

Ответ:

- Потому что он новый и модный
- **Потому что он простой, быстрый и подходит для масштабируемых проектов**
- Потому что его создали в будущем
- Потому что он самый популярный язык в мире

🔗 Шаг 07:

Как называется инструмент для управления пакетами и зависимостями в Go?

Ответ:

- npm
- pip
- **go mod**
- yarn

🔗 Шаг 08:

Что такое пакет (package) в Go?

Ответ:

- Функция
- Отдельный файл
- **Группа функций и типов, объединённых в один модуль**
- Ошибка

🔗 Шаг 09:

Какой файл обычно содержит основную функцию для запуска программы на Go?

Ответ:

- **main.go**
- start.go
- app.go
- run.go

🔗 Шаг 10:

Что нужно сделать, чтобы начать писать на Go?

Ответ:

- **Скачать и установить Go**
- Купить новый компьютер
- Учить другой язык программирования
- Написать первую программу

🔗 Шаг 11:

Что такое «компиляция» в контексте Go?

Ответ:

- Запуск программы
- **Преобразование кода в исполняемый файл**
- Проверка орфографии в коде
- Сохранение проекта

🔗 Шаг 12:

Какую роль играет команда go run ?

Ответ:

- **Компилирует и запускает программу за один шаг**
- Устанавливает Go на компьютер
- Показывает список пакетов
- Открывает редактор кода

🔗 Шаг 13:

Почему Go часто выбирают для разработки серверных приложений?

Ответ:

- Потому что он работает только на Windows
- **Из-за высокой производительности и простоты работы с сетью**
- Потому что это самый старый язык программирования

- Потому что у него красивый интерфейс

🔗 Шаг 14:

Что такое стандартная библиотека в Go?

Ответ:

- **Набор встроенных функций и пакетов, доступных без установки**
- Внешние плагины
- Сборник ошибок
- Документация

🔗 Шаг 15:

Можно ли использовать Go для создания игр?

Ответ:

- Нет, Go только для сайтов
- **Да, существуют библиотеки и фреймворки для разработки игр на Go**
- Нет, Go устарел
- Да, но только для мобильных игр

📖 Урок 10 - Практика Go первые задачи и эксперименты

📖 Урок 11 - Викторина

🔗 Шаг 02:

Вопрос 1

Какая особенность языка Go делает его популярным для создания веб-сервисов?

Подсказка: Подумайте о скорости и простоте.

Ответ:

- **Быстрая компиляция и простая синтаксическая структура**
- Обязательная объектно-ориентированная модель
- Работа только на Windows
- Не поддерживает многопоточность

🔗 Шаг 03:

Вопрос 2 Расположите шаги разработки веб-сервиса на Go в правильном порядке:

1. Разработка логики приложения
2. Настройка сервера и маршрутов
3. Тестирование и отладка
4. Запуск на хостинге

🔗 Подсказка: Сначала создаём код, потом тестируем и только потом деплой.

Ответ:

- 1. Разработка логики приложения
- 2. Настройка сервера и маршрутов
- 3. Тестирование и отладка
- 4. Запуск на хостинге

🔗 Шаг 04:

Вопрос 3

Сопоставьте инструмент Go с его назначением.

Подсказка: Каждый инструмент решает свою задачу.

Ответ:

- **goroutines** — Параллельное выполнение задач
- **channels** — Безопасный обмен данными между потоками
- **net/http** — Создание веб-сервера и обработка запросов
- **templates** — Генерация HTML-страниц для веб-приложений

🔗 Шаг 05:

Вопрос 4

Что из перечисленного является преимуществом использования Go для ботов?

Подсказка: Думайте про масштабируемость и стабильность.

Ответ:

- **Высокая производительность и лёгкая многопоточность**
- Только синхронная работа
- Отсутствие стандартной библиотеки
- Невозможность работы с API

🔗 Шаг 06:

Вопрос 5 Расположите этапы деплоя Go-приложения:

🔗 Подсказка: Подумайте, что нужно делать перед запуском приложения.

Ответ:

- Настройка окружения на сервере
- Загрузка кода на сервер
- Сборка приложения на сервере
- Запуск и проверка работы

🔗 Шаг 07:

Вопрос 6

Сопоставьте тип базы данных с проектом.

Подсказка: Каждый проект использует оптимальный способ хранения данных.

Ответ:

- **MongoDB** — Хранение документов и гибких структур данных
- **PostgreSQL** — Сложные реляционные данные с транзакциями
- **Redis** — Быстрый кэш и временные данные
- **Файловая система** — Простое хранение небольших данных без СУБД

🔗 Шаг 08:

Вопрос 7

Какой инструмент помогает тестировать API Go-приложений?

Подсказка: Подумайте про HTTP-запросы и проверки ответов.

Ответ:

- **Postman**
- Docker
- VS Code
- GitHub

🔗 Шаг 09:

Вопрос 8 Расположите шаги создания Telegram-бота на Go:

1. Регистрация бота через BotFather
2. Настройка вебхука или polling
3. Программирование логики команд
4. Запуск и тестирование бота

🔗 Подсказка: Сначала создаём бота в Telegram, потом программируем и запускаем.

Ответ:

- 1. Регистрация бота через BotFather
- 2. Настройка вебхука или polling
- 3. Программирование логики команд
- 4. Запуск и тестирование бота

🔗 Шаг 10:

Вопрос 9

Сопоставьте тип теста с его задачей.

Подсказка: Каждый тест проверяет свой аспект работы приложения.

Ответ:

- **Unit-тест** — Проверка отдельной функции
- **Integration-тест** — Проверка взаимодействия нескольких модулей
- **End-to-end тест** — Проверка всей системы от начала до конца
- **Load-тест** — Проверка приложения под нагрузкой

🔗 Шаг 11:

Вопрос 10

Что важно учитывать при работе с Go и внешними API?

Подсказка: Думайте про ошибки и ограничения API.

Ответ:

- **Обрабатывать ошибки и ограничения запросов**
- Игнорировать ответы API
- Отправлять запросы без таймаутов
- Не использовать структуры для JSON

Модуль 2 - Модуль 2. Основы Go без скучной теории

Урок 1 - Переменные и типы данных на примерах

Шаг 02:

Вопрос 1 — Знакомство с ящиком

В программировании переменная — это:

Ответ:

- Место, где лежит код
- **Подписанный ящик, где хранится значение**
- Кнопка запуска программы

Шаг 03:

Вопрос 2 — Имя героя

В коде:

```
var age int = 15
```

Как называется переменная?

Ответ:

- int
- **15**
- age

Шаг 04:

Вопрос 3 — Сколько яблок?

Какой тип данных подойдёт для количества яблок?

Ответ:

- **int**
- string
- bool
- 5002

Шаг 05:

Вопрос 4 — Число с запятой

В Go число 3.14 лучше хранить в:

Ответ:

- int
- **float64**
- bool

Шаг 06:

Вопрос 5 — Текстовое заклинание

Какой тип данных хранит фразу "Привет" ?

Ответ:

- **string**
- bool
- float32

Шаг 07:

Вопрос 6 — Да или нет?

Что может хранить переменная типа bool ?

Ответ:

- **Только true или false**
- Любое число
- Любой текст

Шаг 08:

Вопрос 7 — Автоматический выбор

Что произойдёт при записи:

```
name := "Аня"
```

Ответ:

- **Go автоматически присвоит тип string**
- Go выдаст ошибку
- Go присвоит тип int

Шаг 09:

Что случится в коде:

```
var age int = 15  
age = "пятнадцать"
```

Ответ:

- Программа выполнится, но изменит тип
- **Go выдаст ошибку**
- Go превратит текст в число

Шаг 10:

Вопрос 9 — Меняющийся ящик

Какое значение будет у переменной score после выполнения?

```
var score int = 0  
score = 10  
score = score + 5
```

Ответ:

- 0
- 10
- **15**

🔧 Шаг 11:

Вопрос 10 — Что такое тип данных?

Тип данных — это:

Ответ:

- Скорость работы программы
- **Вид содержимого переменной**
- Название файла с кодом

📖 Урок 2 - Математика в Go калькулятор за 15 минут

🔧 Шаг 02:

Вопрос 1 — Магическая машинка

Go в этой истории сравнивается с:

Ответ:

- Калькулятором в телефоне
- **Волшебной машинкой, которая выполняет команды**
- Роботом-пылесосом

🔧 Шаг 03:

Вопрос 2 — Первая деталь конструктора

В калькуляторе переменные num1 и num2 нужны для:

Ответ:

- **Хранения введённых чисел**
- Запоминания имени пользователя
- Выбора математической операции

🔧 Шаг 04:

Вопрос 3 — Сканер данных

Что делает строка:

```
fmt.Scanln(&num1)
```

Ответ:

- Печатает число
- **Ждёт, пока пользователь введёт число**
- Удаляет переменную

🔧 Шаг 05:

Вопрос 4 — Испытание делением

Что произойдёт, если ввести / и второе число = 0?

Ответ:

- Go выполнит деление
- **Появится сообщение «Ошибка: деление на ноль!»**
- Программа закроется без ответа

🧙 Шаг 06:

Вопрос 5 — Четыре силы калькулятора

Какие операции из списка умеет выполнять этот калькулятор? а) +, -, *, / б) +, -, ^, √

Ответ:

- +, -, *, /
- +, -, ^, √
- Только сложение

🧙 Шаг 07:

Вопрос 6 — Выбор заклинания

Какая переменная хранит знак операции?

Ответ:

- operation
- result
- fmt

🧙 Шаг 08:

Вопрос 7 — Прокачка калькулятора

Какое улучшение НЕ упомянуто в статье? а) Возведение в степень б) Работа с несколькими операциями подряд с) Построение графика функций

Ответ:

- Возведение в степень
- Работа с несколькими операциями подряд
- **Построение графика функций**

🧙 Шаг 09:

Вопрос 8 — LEGO-логика

Почему на уроке программирование сравнивается с LEGO?

Ответ:

- Детали нужно покупать
- **Можно строить сложное из простых элементов**
- Оно пластиковое

🧙 Шаг 10:

Вопрос 9 — Финальная проверка

Что делает этот фрагмент?

```
if operation == "+" {  
    result = num1 + num2  
}
```

Ответ:

- **Проверяет, выбрал ли пользователь сложение, и выполняет его**
- Считает сумму всегда, без проверки
- Проверяет только второе число

Урок 3 - Условия и логика мини-игра «Угадай число»

Шаг 02:

Миссия 1. Что такое условие?

Когда ты принимаешь решение — это условие .

 Пример из жизни:

Если на улице дождь — возьми зонт. Иначе — иди без зонта.

Так же и в коде!

Задание 1:

Заполни пропуски в коде, чтобы он работал:

```
if температура > 30 {  
    fmt.Println("Очень жарко!")  
} else {  
    fmt.Println("___ прохладно!")  
}
```

 Вопрос:

Ответ:

- Что напечатает программа, если температура = 25 ?
- Очень жарко!
- Наверное, прохладно!
- **Просто прохладно!**  Подсказка: 25 не больше 30 → будет else

Шаг 03:

Миссия 2. Проверяем числа

Теперь мы будем сравнивать числа и решать, что делать.

Задание 2:

Напиши программу, которая говорит:

Ответ:

- « Всё хорошо», если число больше 0
- « Ошибка», если число меньше или равно 0

```
var число int  
fmt.Print("Введи число: ")  
fmt.Scan(&число)  
  
if число > 0 {  
    fmt.Println(" Всё хорошо")  
} else {  
    fmt.Println(" Ошибка")  
}
```

 Вопрос:

- Что напечатает программа, если ввести -5?
-  Всё хорошо
-  **Ошибка**

🔧 Шаг 04:

🎯 Миссия 3. Случайное число

Компьютер может сам придумать число!

```
import (
    "math/rand"
    "time"
)

rand.Seed(time.Now().UnixNano())
число := rand.Intn(100) + 1
```

📄 Вопрос:

Ответ:

- Что делает `rand.Intn(100) + 1` ?
- **Даёт число от 1 до 100**
- Даёт число от 0 до 100
- Даёт число от 1 до 99

📖 Урок 4 - Циклы автоматизация действий

🔧 Шаг 02:

🎯 Миссия 1. Повторять до победы

Мы хотим спрашивать число до тех пор, пока игрок не угадает .

Задание 3:

Какой кусок кода делает повторение ?

а) `if` б) `for { }` в) `break`

✓ Правильный ответ: б) `for { }`

Ответ:

- `if`
- **`for { }`**
- `break`

🔧 Шаг 03:

🎯 Миссия 2: Попугай-программист

📖 Задание: Напиши программу, которая 10 раз повторяет фразу:

Я больше не забуду поставить точку с запятой.

🔧 Объяснение: Цикл `for` повторяет команду много раз. Укажем, что нужно 10 повторов.

✓ Код:

```
for i := 1; i <= 10; i++ {
    fmt.Println("Я больше не забуду поставить точку с запятой.")
}
```

🗣 Вопрос: Сколько раз напечатается фраза?

Ответ:

- 9
- **10**
- 11

🔔 Шаг 04:

📋 Миссия 3: Считаем от 1 до 5

📋 Задание: Напиши программу, которая выводит числа от 1 до 5.

✓ Код:

```
for i := 1; i <= 5; i++ {  
    fmt.Println(i)  
}
```

🔗 Результат:

1 2 3 4 5

🤖 Вопрос: Что будет, если заменить i++ на i += 2 ?

Ответ:

- 1 2 3 4 5
- **1 3 5**
- ошибка

🔔 Шаг 05:

📋 Миссия 4: Обратный отсчёт

📋 Задание: Напиши таймер, который считает от 10 до 1, а потом пишет «Старт!»

✓ Код:

```
for i := 10; i >= 1; i-- {  
    fmt.Println(i)  
}  
fmt.Println("Старт!")
```

🔗 Результат:

10
9
8
...
1
Старт!

🤖 Вопрос: Что делает i-- ?

Ответ:

- Увеличивает число
- **Уменьшает число**
- Умножает число

🔔 Шаг 06:

★ Миссия 5: Только чётные

📋 Задание: Выведи все чётные числа от 1 до 20.

💡 Чётные — это те, которые делятся на 2 без остатка.

✓ Код:

```
for i := 1; i <= 20; i++ {  
    if i%2 == 0 {  
        fmt.Println(i)  
    }  
}
```

🔑 Результат:

```
2  
4  
6  
...  
20
```

🤔 Вопрос: Что значит `i % 2 == 0` ?

Ответ:

- Число нечётное
- **Число чётное**

📖 Урок 5 - Функции делаем код умнее

💡 Шаг 02:

🎯 Миссия 1: «Привет, друг!»

Вопрос: Что делает функция `sayHello(name string)` в этом коде?

```
func sayHello(name string) {  
    fmt.Println("Привет,", name)  
}
```

Ответ:

- Умножает два числа
- **Печатает приветствие с именем**
- Считает длину имени

💡 Шаг 03:

🎯 Миссия 2: «Функция — это...»

Вопрос: Что такое функция в программировании?

Ответ:

- Комментарий в коде
- Таблица умножения
- **Набор команд с именем, который можно запускать**

💡 Шаг 04:

🎯 Миссия 3: «Считаем квадрат»

Вопрос: Что вернёт функция `square(4)` , если она написана так:

```
func square(n int) int {  
    return n * n  
}
```

Ответ:

- 8
- **16**
- 4
- 5002

🔮 Шаг 05:

+ Миссия 4: «Что вернёт калькулятор?»

Вопрос: Что выведет этот код?

```
func add(a int, b int) int {  
    return a + b  
}
```

```
func main() {  
    fmt.Println(add(3, 5))  
}
```

Ответ:

- 35
- **8**
- 15

🔮 Шаг 06:

Миссия 5: «Зачем нужны функции?»

Вопрос: Зачем программисты используют функции?

Ответ:

- Чтобы сделать код длиннее
- Чтобы повторять один и тот же код снова
- **Чтобы не повторяться и сделать код понятнее**

🔮 Шаг 07:

🍷 Миссия 6: «Готовим пиццу»

Вопрос: Что выведет этот код?

```
func makePizza(topping string) {  
    fmt.Println("Пицца с", topping)  
}
```

```
func main() {  
    makePizza("сыром")  
}
```

Ответ:

- Пицца с тестом
- **Пицца с сыром**

- Пицца без начинки

🔗 Шаг 08:

Миссия 7: «Что делает return?»

Вопрос: Для чего используется return в функции?

Ответ:

- Чтобы остановить программу
- **Чтобы вернуть результат из функции**
- Чтобы начать новую функцию

📖 Урок 6 - Викторина 1 основы Go

🔗 Шаг 02:

Как объявить переменную age типа int в Go?

Ответ:

- var age = "int"
- **var age int**
- int age
- age := int

🔗 Шаг 03:

Что выведет код?

```
var name string = "Alex"
fmt.Println(name)
```

Ответ:

- **Alex**
- "Alex"
- name
- Ошибка компиляции

🔗 Шаг 04:

Какой тип данных в Go используется для хранения текста?

Ответ:

- int
- bool
- **string**
- float64

🔗 Шаг 05:

Как правильно записать короткое объявление переменной score со значением 100?

Ответ:

- score int = 100
- **score := 100**
- var score := 100

- `var score int := 100`

🔗 Шаг 06:

Математика в Go

1. Что выведет код?

```
fmt.Println(7 + 3*2)
```

Ответ:

- 20
- **13**
- 17
- 10

🔗 Шаг 07:

Какой тип переменной подойдёт для дробных чисел?

Ответ:

- int
- **float64**
- string
- bool

🔗 Шаг 08:

Как правильно найти остаток от деления 10 на 3 ?

Ответ:

- **10 % 3**
- 10 / 3
- 10 * 3
- остаток(10, 3)

🔗 Шаг 09:

1. Что выведет код?

```
x := 5
```

```
x += 2
```

```
fmt.Println(x)
```

Ответ:

- 5
- 2
- **7**
- Ошибка

🔗 Шаг 10:

Условия и логика

Как записать условие: «Если x больше 10»?

Ответ:

- if x => 10 {
- if x = 10 {
- **if x > 10 {**
- if (x больше 10) {

 **Шаг 11:**

1. Что выведет код?

```
num := 7
if num%2 == 0 {
    fmt.Println("Чётное")
} else {
    fmt.Println("Нечётное")
}
```

Ответ:

- Чётное
- **Нечётное**
- 7
- Ошибка

 **Шаг 12:**

Какое логическое выражение вернёт true , если a меньше 5 и b больше 10?

Ответ:

- a < 5 || b > 10
- **a < 5 && b > 10**
- a > 5 && b < 10
- a <= 5 && b <= 10

 **Шаг 13:**

Что означает оператор != в Go?

Ответ:

- Присвоить значение
- Меньше или равно
- **Не равно**
- И

 **Шаг 14:**

Какой цикл в Go чаще всего используется для повторений?

Ответ:

- while
- **for**
- repeat

- loop

🔔 Шаг 15:

1. Что выведет код?

```
for i := 1; i <= 3; i++ {  
    fmt.Println(i)  
}
```

Ответ:

- 1 2
- 2 3
- **1 2 3**
- 0 1 2 3

🔔 Шаг 16:

Как завершить цикл досрочно?

Ответ:

- stop
- quit
- **break**
- breek

📖 Урок 7 - Викторина 2 основы Go

🔔 Шаг 02:

Как пропустить одну итерацию цикла?

Ответ:

- stop
- skip
- **continue**
- pass

🔔 Шаг 03:

Как объявить функцию sayHello , которая ничего не принимает и не возвращает?

Ответ:

- **func sayHello() { // код }**
- sayHello() func { // код }
- func sayHello { // код }
- function sayHello() { // код }

🔔 Шаг 04:

1. Что выведет код?

```
func sum(a int, b int) int {  
    return a + b  
}
```

```
fmt.Println(sum(3, 4))
```

Ответ:

- 34
- **7**
- sum(3,4)
- Ошибка

Шаг 05:

Что означает ключевое слово return в Go?

Ответ:

- Вывести текст на экран
- **Вернуть значение из функции**
- Завершить программу
- Объявить переменную

Шаг 06:

Как вызвать функцию greet() , чтобы она выполнялась?

Ответ:

- greet
- **greet()**
- func greet
- call greet

Урок 8 - Задачи для закрепления

Урок 9 - Викторина

Шаг 02:

Вопрос 1

Что делает язык Go особенно удобным для новичков при создании программ?

Подсказка: Подумайте про синтаксис и компиляцию.

Ответ:

- **Простой и понятный синтаксис**
- Обязательная объектно-ориентированность
- Только интерпретируемый код
- Невозможность работы с веб

Шаг 03:

Вопрос 2 Расположите шаги создания простой программы на Go:

1. Написание кода функций
2. Компиляция программы
3. Запуск и проверка работы
4. Исправление ошибок

Подсказка: Сначала кодим, потом проверяем и исправляем ошибки.

Ответ:

- 1. Написание кода функций
- 2. Компиляция программы
- 3. Запуск и проверка работы
- 4. Исправление ошибок

🔮 Шаг 04:

Вопрос 3

Сопоставьте тип данных Go с его примером.

Подсказка: Думайте про числа, текст и логические значения.

Ответ:

- **int** — 42
- **float64** — 3.14
- **string** — "Привет"
- **bool** — true

🔮 Шаг 05:

Вопрос 4

Что делает оператор `:=` в Go?

Подсказка: Это сочетание объявления и присвоения.

Ответ:

- **Объявляет переменную и сразу присваивает значение**
- Только сравнивает значения
- Только присваивает значение существующей переменной
- Удаляет переменную

🔮 Шаг 06:

Вопрос 5 Расположите шаги при работе с массивами в Go:

1. Объявление массива
2. Инициализация значений
3. Доступ к элементам
4. Обработка элементов (например, суммирование)

Подсказка: Сначала создаём массив, потом с ним работаем.

Ответ:

- 1. Объявление массива
- 2. Инициализация значений
- 3. Доступ к элементам
- 4. Обработка элементов (например, суммирование)

🔮 Шаг 07:

Вопрос 6

Сопоставьте оператор Go с его действием.

Подсказка: Каждый оператор решает конкретную задачу.

Ответ:

- + — Сложение чисел
- - — Вычитание
- * — Умножение
- / — Деление

🔗 **Шаг 08:**

Вопрос 7

Что делает функция `len()` в Go?

Подсказка: Подумайте, что она может вернуть для строки или массива.

Ответ:

- **Возвращает количество элементов или символов**
- Возвращает последний элемент массива
- Удаляет элемент массива
- Создаёт копию массива

🔗 **Шаг 09:**

Вопрос 8 Расположите шаги создания простой функции в Go:

1. Объявление функции
2. Добавление параметров
3. Описание тела функции
4. Вызов функции в коде

Подсказка: Сначала описываем функцию, потом вызываем её.

Ответ:

- 1. Объявление функции
- 2. Добавление параметров
- 3. Описание тела функции
- 4. Вызов функции в коде

🔗 **Шаг 10:**

Вопрос 9

Сопоставьте тип цикла Go с примером его использования.

Подсказка: Каждый цикл используется в разных ситуациях.

Ответ:

- **for** — Повторять действия определённое число раз
- **for range** — Итерировать по элементам массива или среза
- **while (эмуляция через for)** — Повторять пока условие истинно

🔗 **Шаг 11:**

Вопрос 10

Что важно учитывать при работе с функциями и переменными в Go?

Подсказка: Думайте про область видимости и возврат значений.

Ответ:

- **Область видимости и правильный возврат значений**
- Можно использовать переменные из любой функции без объявления
- Функции не могут возвращать значения
- Все переменные глобальные по умолчанию

Модуль 3 - Модуль 3. Работа с данными и файлами

Урок 1 - Массивы и срезы хранение списков

Шаг 02:

Миссия 1: «Шкаф на 5 полок»

Ситуация: Ты собираешь коллекцию оценок. Тебе нужно место ровно на 5 штук.

```
var scores [5]int
```

Вопрос: Что означает [5]int в этом объявлении?

Ответ:

- Массив из 5 строк
- Срез с неизвестным количеством элементов
- **Массив из 5 целых чисел**

Шаг 03:

Миссия 2: «Добавь новую игру!»

Ситуация: У тебя уже есть список игр:

```
games := []string{"Minecraft", "Roblox", "CS:GO"}
```

Ты хочешь добавить "Valorant".

Какой код подойдёт?

Ответ:

- games[3] = "Valorant"
- **append(games, "Valorant")**
- games.add("Valorant")

Шаг 04:

Миссия 3: «Что выбрать?»

Вопрос: В чём главное отличие массива от среза ?

Ответ:

- **Массив можно изменять, срез — нет**
- **Срез** — это список фиксированной длины
- **Массив** — фиксированный по размеру, срез — гибкий

Шаг 05:

Миссия 4: «Средний балл»

Ты собираешь 3 оценки от пользователя и сохраняешь их в срез:

```
var grades []int
```

Вопрос: Какая команда добавит в grades новую оценку?

Ответ:

- grades + grade
- **grades = append(grades, grade)**
- grades.insert(grade)

🔮 **Шаг 06:**

Миссия 5: «Список покупок»

У тебя есть список:

```
shoppingList := []string{"Хлеб", "Молоко"}
```

Ты добавляешь:

```
shoppingList = append(shoppingList, "Шоколад", "Чипсы")
```

Вопрос: Что будет в shoppingList[2] ?

Ответ:

- "Молоко"
- **"Шоколад"**
- "Чипсы"

Урок 2 - Map быстрый поиск по ключу

🔮 **Шаг 02:**

Вступление (Сюжет)

Ты оказался в цифровом мире, где все знания хранятся в магических Картах . Только зная, как пользоваться этими картами, ты сможешь находить предметы, восстанавливать утерянные данные и спасти этот мир от хаоса.

Миссия 1: Создание карты

🎯 **Задача:** Создать карту, в которой ключи — названия предметов, а значения — их количество.

Пример кода:

```
инвентарь := make(map[string]int)
инвентарь["Меч"] = 1
инвентарь["Зелье"] = 3
```

? Вопрос 1:

Что делает строка инвентарь := make(map[string]int) ?

Ответ:

- Создаёт список предметов
- **В карту добавится ключ "Зелье" с числом 3**
- Ошибка компиляции
- Создастся переменная

🔮 Шаг 03:

Миссия 2: Поиск по ключу

📖 Задача: Узнать, сколько зелья у тебя осталось.

```
fmt.Println("Зелья осталось:", инвентарь["Зелье"])
```

? Вопрос 3:

Что произойдёт, если ты попытаешься получить значение по ключу, которого нет в карте?

Ответ:

- Вернётся 0
- Произойдёт сбой
- Go создаст новый ключ
- Вернётся "undefined"

🔮 Шаг 04:

Миссия 3: Проверка наличия

📖 Задача: Проверить, есть ли в инвентаре "Щит".

```
значение, есть := инвентарь["Щит"]
if есть {
    fmt.Println("Щит есть:", значение)
} else {
    fmt.Println("Щита нет")
}
```

? Вопрос:

Что означает переменная есть в этом коде?

Ответ:

- Число предметов
- Имя переменной
- **Истина/ложь** — есть ли такой ключ
- Количество ключей в карте

🔮 Шаг 05:

Миссия 4: Изменение значения

✂ Задача: Изменить количество зелья на 5.

```
инвентарь["Зелье"] = 5
```

? Вопрос 5:

Если ключ уже есть в карте и мы присваиваем ему новое значение...

Ответ:

- Ключ удалится
- Ничего не изменится
- **Значение будет обновлено**
- Go выдаст ошибку

🔧 Шаг 06:

Миссия 5: Удаление

📖 Задача: Удалить "Меч" из инвентаря.

```
delete(инвентарь, "Меч")
```

? Вопрос 6:

Что делает функция delete ?

Ответ:

- Обнуляет значение
- Удаляет карту целиком
- **Удаляет указанный ключ**
- Выводит сообщение об ошибке

🔧 Шаг 07:

Миссия 6: Перебор карты

🔗 Задача: Показать весь инвентарь.

```
for предмет, количество := range инвентарь {  
    fmt.Println(предмет, ":", количество)  
}
```

? Вопрос:

Что делает цикл for ... range с картой?

Ответ:

- Удаляет все элементы
- **Перебирает и выводит пары ключ-значение**
- Сортирует карту
- Преобразует карту в массив

🔧 Шаг 08:

🚩 Финальный тест: Мини-проект

📖 Задача: Ты создаёшь карту телефонная_книга , где:

Ответ:

- Ключ: имя друга
- Значение: номер телефона

```
телефонная_книга := make(map[string]string)  
телефонная_книга["Аня"] = "+7 999 111-22-33"  
телефонная_книга["Максим"] = "+7 912 555-77-88"
```

? Вопрос 8:

Какой тип у этой карты?

- map[int]string
- **map[string]string**
- map[string]int

- map[int]int

Урок 3 - Чтение и запись файлов свой блокнот

Шаг 02:

 Представь, что ты получил волшебный артефакт — электронный блокнот . Ты можешь писать в него мысли, планы и идеи, а потом возвращаться к ним в любой момент.

Чтобы использовать блокнот, тебе нужно научиться работать с файлами в языке Go. Готов? Погнали! Миссия 1: Что такое файл?

 Файл — это место, где твоя программа может хранить информацию: текст, числа, рецепты, секретные коды...

? Вопрос 1:

Что такое текстовый файл?

Ответ:

- Программа
- Место в интернете
- **Хранилище текстовой информации**
- Команда в терминале

Шаг 03:

Миссия 2: Первая запись в файл

Ты решил записать первую заметку в блокнот. В Go это делается с помощью функции `os.WriteFile` .

```
text := "Привет, это моя первая заметка!"  
err := os.WriteFile("note.txt", []byte(text), 0644)
```

? Вопрос:

Что делает функция `os.WriteFile` ?

Ответ:

- Читает файл
- Удаляет файл
- **Записывает данные в файл**
- Показывает текст на экране

Шаг 04:

Миссия 3: Чтение файла

Теперь ты хочешь прочитать, что записал. Это можно сделать так:

```
data, err := os.ReadFile("note.txt")  
fmt.Println(string(data))
```

? Вопрос 4:

Что делает `os.ReadFile("note.txt")` ?

Ответ:

- Удаляет файл
- Открывает файл и записывает в него

- Читает содержимое файла
- Создаёт новый файл

🔗 Шаг 05:

Миссия 4: Собираем свой блокнот

Ты создаёшь настоящее приложение "Мой блокнот" ! Оно:

Ответ:

- Позволяет добавить заметку
- Сохраняет её в файл
- Читает заметки обратно

🔗 Используем `os.OpenFile` для записи в файл без удаления старых заметок :

```
file, err := os.OpenFile("notes.txt", os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0644)
file.WriteString(note + "\n")
```

Вопрос :

Что делает флаг `os.O_APPEND` ?

- Заменяет старый файл
- Записывает в начало файла
- **Добавляет текст в конец файла**
- Читает файл

🔗 Шаг 06:

Миссия 5: Интерфейс блокнота

Ты сделал меню:

```
fmt.Println("1. Добавить заметку")
fmt.Println("2. Прочитать заметки")
fmt.Println("3. Выход")
```

? Вопрос 7:

Что делает `bufio.NewReader(os.Stdin)` ?

Ответ:

- Пишет файл
- Читает данные из файла
- **Читает ввод пользователя**
- Завершает программу

🔗 Шаг 07:

🏁 Финальная миссия: Улучши блокнот

🔗 Ты можешь улучшить своё приложение:

Ответ:

- Добавить дату к каждой заметке
- Делать поиск по тексту

- Записывать каждую заметку в отдельный файл

Вопрос 9:

Какая идея может улучшить блокнот?

- Записывать заметки в интернет
- Добавить случайное удаление заметок
- **Добавлять дату и время к каждой заметке**
- Скрыть заметки навсегда

Урок 4 - JSON язык данных интернета

Шаг 02:

Ты оказался в цифровом мире, где все данные — от сообщений в чате до погоды на улице — передаются в особом формате — JSON . Этот язык понимают все : сайты, приложения, даже холодильники 😊

Твоя задача — освоить работу с JSON и спасти данные интернета!

Миссия 1: Что такое JSON?

 Тебе попался странный текст:

```
{
  "name": "Иван",
  "age": 15,
  "hobbies": ["Go", "Шахматы", "Футбол"],
  "is_student": true
}
```

? Вопрос:

Что это такое?

Ответ:

- HTML-код
- Таблица Excel
- **JSON** — формат для хранения и обмена данными
- Исходный код игры

Шаг 03:

Миссия 2: Из чего состоит JSON?

Ты изучаешь структуру JSON. Вот пример:

```
{
  "name": "Аня",
  "age": 14,
  "hobbies": ["Рисование", "Go", "Чтение"]
}
```

? Вопрос:

Что означает "hobbies": ["Рисование", "Go", "Чтение"] ?

Ответ:

- Строка

- Число
- **Список (массив)**
- Команда

🔗 Шаг 04:

Миссия 3: Превращаем Go-структуру в JSON

Ты создаёшь Go-программу, которая превращает структуру в JSON:

```
type Person struct {
    Name    string    `json:"name"`
    Age     int       `json:"age"`
    Hobbies []string  `json:"hobbies"`
}
```

? Вопрос:

Зачем нужны теги `json:"name"` в структуре Go?

Ответ:

- Это просто комментарий
- **Указывают, как поле будет называться в JSON**
- Это имя переменной
- Для сортировки полей

🔗 Шаг 05:

Миссия 4: Чтение JSON в Go

Ты получил от сервера JSON:

```
data := `{"name": "Илья", "age": 16, "hobbies": ["Go", "Баскетбол"]}`
```

Ты хочешь превратить это в структуру Go с помощью:

```
err := json.Unmarshal([]byte(data), &person)
```

? Вопрос:

Что делает `json.Unmarshal`?

Ответ:

- Записывает в JSON
- Удаляет JSON
- **Преобразует JSON в структуру Go**
- Сортирует данные

🔗 Шаг 06:

Миссия 5: Работаем с JSON-файлом

В файле `friends.json` лежат данные:

```
[
  {"name": "Маша", "age": 15},
  {"name": "Петя", "age": 16},
```

```
    {"name": "Саша", "age": 14}  
]
```

Ты читаешь их в Go:

```
var friends []Friend  
err = json.Unmarshal(data, &friends)
```

? Вопрос:

Что означает []Friend ?

Ответ:

- Один друг
- **Массив (срез) структур Friend**
- Неправильный синтаксис
- Ключ-значение

🔧 Шаг 07:

📁 Финальная миссия: Список друзей

Ты хочешь напечатать список друзей из JSON-файла.

```
for _, f := range friends {  
    fmt.Printf("%s (%d лет)\n", f.Name, f.Age)  
}
```

? Вопрос:

Что делает цикл for _, f := range friends ?

Ответ:

- Считает длину массива
- Преобразует JSON
- **Перебирает всех друзей в массиве**
- Удаляет элементы

📖 Урок 5 - Мини-проект список друзей с сохранением

🔧 Шаг 02:

★ Миссия 1: Знакомство с заданием

Представь, что ты записываешь друзей в тетрадь. Но что если она порвётся или потеряется? 🤔

📁 Вместо бумаги мы используем JSON-файл , а писать туда будем с помощью Go .

? Вопрос 1:

Зачем сохранять список друзей в файл?

Ответ:

- Чтобы память компьютера не перегружилась
- **Чтобы данные не исчезли при выключении программы**
- Просто потому что Go так требует
- Чтобы шифровать список

🔧 Шаг 03:

🔪 Миссия 2: Готовим инструменты

Создай папку и файл:

```
mkdir friendslist
cd friendslist
touch main.go
```

Импортируем пакеты:

```
import (
    "encoding/json"
    "fmt"
    "os"
    "bufio"
    "strings"
)
```

? Вопрос 2:

Зачем нам пакет encoding/json ?

Ответ:

- Для работы с изображениями
- Для вывода на экран
- **Для работы с JSON-данными**
- Для подключения интернета

🔧 Шаг 04:

Миссия 3: Создаём структуру

```
type Friend struct {
    Name string `json:"name"`
}
```

? Вопрос:

Что такое Friend ?

Ответ:

- Массив
- Тип переменной
- **Структура с полем Name**
- Команда добавления

🔧 Шаг 05:

📁 Миссия 4: Загружаем друзей из файла

```
func loadFriends(filename string) ([]Friend, error) {
    var friends []Friend

    file, err := os.ReadFile(filename)
    if err != nil {
        return friends, nil // если файла нет, возвращаем пустой список
    }
}
```

```

    }

    err = json.Unmarshal(file, &friends)
    return friends, err
}

```

? Вопрос 4:

Что произойдёт, если файл не найден?

Ответ:

- Программа завершится
- Вернётся ошибка
- **Вернётся пустой список**
- Друзья удалятся

🔗 Шаг 06:

📁 Миссия 5: Сохраняем список

```

func saveFriends(filename string, friends []Friend) error {
    data, err := json.MarshalIndent(friends, "", " ")
    if err != nil {
        return err
    }
    return os.WriteFile(filename, data, 0644)
}

```

? Вопрос:

Что делает json.MarshalIndent ?

Ответ:

- Создаёт таблицу
- Удаляет файл
- **Преобразует данные в читаемый JSON**
- Загружает HTML

🔗 Шаг 07:

🧠 Миссия 6: Добавление друга

```

name, _ := reader.ReadString('\n')
name = strings.TrimSpace(name)
friends = append(friends, Friend{Name: name})
saveFriends(filename, friends)

```

? Вопрос:

Что делает append ?

Ответ:

- Удаляет друга
- **Добавляет нового друга в список**
- Сохраняет файл
- Загружает JSON

Урок 6 - Викторина работа с данными

Шаг 02:

Что такое массив в Go?

Ответ:

- **Упорядоченный набор элементов фиксированной длины**
- Неупорядоченный набор элементов
- Коллекция элементов переменной длины
- Тип данных для хранения только строк

Шаг 03:

Как объявить срез из целых чисел?

Ответ:

- **var s []int**
- var s int[]
- s := array{1, 2}
- s := (1, 2, 3)

Шаг 04:

Можно ли менять размер массива после объявления?

Ответ:

- Да, легко добавлять элементы
- **Нет, размер фиксирован**
- Да, если использовать функцию append
- Да, если массив содержит строки

Шаг 05:

Что вернёт len(s) для среза s?

Ответ:

- Количество байт в срезе
- **Количество элементов в срезе**
- Максимальный размер среза
- Индекс последнего элемента

Шаг 06:

Что такое map в Go?

Ответ:

- **Структура данных для ключ-значение**
- Список ключей
- Массив чисел
- Специальный вид среза

Шаг 07:

Как объявить пустую карту из string в int?

Ответ:

- **var m map[string]int**
- var m map[int]string
- m := map[int]int{}
- m := map{}

🔗 Шаг 08:

Как получить значение по ключу из карты?

Ответ:

- **m.get("ключ")**
- m["ключ"]
- get(m, "ключ")
- m.key("ключ")

🔗 Шаг 09:

Что произойдёт, если запросить несуществующий ключ в map?

Ответ:

- Ошибка компиляции
- **Возвращается нулевое значение типа**
- Возвращается пустая строка
- Программа зависнет

🔗 Шаг 10:

Как открыть файл для чтения?

Ответ:

- os.Create()
- **os.Open()**
- ioutil.ReadFile()
- file.Open()

🔗 Шаг 11:

Как записать строку в файл?

Ответ:

- Использовать fmt.Println
- **Использовать file.Write или file.WriteString**
- Использовать os.WriteString
- Использовать map.Write

🔗 Шаг 12:

Что нужно делать после работы с файлом?

Ответ:

- Ничего
- **Закрыть файл методом Close()**
- Сохранить файл вручную

- Вызывать функцию flush()

🔗 Шаг 13:

Какая библиотека в Go подходит для работы с файлами?

Ответ:

- net/http
- **os**
- fmt
- json

🔗 Шаг 14:

Что такое JSON?

Ответ:

- Язык программирования
- **Формат хранения данных**
- Тип файла для изображений
- Способ сортировки

🔗 Шаг 15:

Что такое десериализация JSON?

Ответ:

- **Преобразование JSON в структуру Go**
- Запись JSON в файл
- Сжатие JSON
- Отправка JSON по сети

🔗 Шаг 16:

Для чего чаще всего используют JSON?

Ответ:

- **Для хранения конфигураций и обмена данными между сервисами**
- Для создания сайтов
- Для рисования графики
- Для обработки звука

📖 Урок 7 - Задачи для закрепления

📖 Урок 8 - Викторина

🔗 Шаг 02:

Вопрос 1

Что в Go используется для чтения и записи файлов?

Подсказка: Смотрите на стандартную библиотеку.

Ответ:

- **Пакет os**
- Пакет fmt

- Пакет math
- Пакет time

🔗 Шаг 03:

Вопрос 2 Расположите шаги для открытия и записи файла в Go:

1. Открытие файла с нужными правами
2. Запись данных в файл
3. Закрытие файла после работы
4. Обработка ошибок

Подсказка: Сначала открываем файл, потом работаем с ним, не забываем закрыть.

Ответ:

- 1. Открытие файла с нужными правами
- 2. Запись данных в файл
- 3. Закрытие файла после работы
- 4. Обработка ошибок

🔗 Шаг 04:

Вопрос 3

Сопоставьте тип данных с примером, который может храниться в файле.

Подсказка: Думайте про текст, числа и логические значения.

Ответ:

- **string** — "Привет, мир"
- **int** — 42
- **float64** — 3.14
- **bool** — true

🔗 Шаг 05:

Вопрос 4

Что делает функция `ioutil.ReadFile``?

Подсказка: Она позволяет сразу получить содержимое файла.

Ответ:

- **Считывает весь файл в память**
- Создаёт новый файл
- Удаляет файл
- Закрывает файл без чтения

🔗 Шаг 06:

Вопрос 5 Расположите шаги обработки CSV-файла в Go:

1. Открыть CSV-файл
2. Создать Reader
3. Считать строки по очереди
4. Закрыть файл

Подсказка: Работа с CSV похожа на обычное чтение файла, но с Reader.

Ответ:

- 1. Открыть CSV-файл
- 2. Создать Reader
- 3. Считать строки по очереди
- 4. Заккрыть файл

 Шаг 07:

Вопрос 6

Сопоставьте методы Go для работы с файлами с их назначением.

Подсказка: Каждый метод выполняет определённое действие.

Ответ:

- **os.Open** — Открывает файл для чтения
- **os.Create** — Создаёт новый файл
- **file.Close** — Закрывает файл
- **file.Write** — Записывает данные в файл

 Шаг 08:

Вопрос 7

Что нужно сделать после записи данных в файл?

Подсказка: Без этого изменения могут не сохраниться.

Ответ:

- **Заккрыть файл**
- Повторно открыть файл
- Удалить файл
- Ничего не делать

 Шаг 09:

Вопрос 8 Расположите шаги обработки JSON-файла в Go:

1. Открыть файл
2. Создать Decoder
3. Декодировать данные в структуру
4. Заккрыть файл

Подсказка: JSON требует Decoder для разбора структур.

Ответ:

- 1. Открыть файл
- 2. Создать Decoder
- 3. Декодировать данные в структуру
- 4. Заккрыть файл

 Шаг 10:

Вопрос 9

Сопоставьте формат файла с примером использования:

Подсказка: Каждый формат удобен для своей задачи.

Ответ:

- **CSV** — Табличные данные
- **JSON** — Структурированные данные для API
- **TXT** — Простой текст
- **XML** — Данные с вложенной структурой

🔔 **Шаг 11:**

Вопрос 10

Что важно при работе с большими файлами в Go?

Подсказка: Не стоит загружать весь файл в память.

Ответ:

- **Чтение файла по частям (streaming)**
- Сразу загрузить весь файл в память
- Игнорировать ошибки чтения
- Использовать только глобальные переменные

📁 **Модуль 4 - Модуль 4. Интернет и Go**

📖 **Урок 1 - Что такое API и как с ним работать**

🔔 **Шаг 02:**

Квест: «Операция API: Спаси данные!»

📖 **Вступление:**

Ты — агент цифрового спецотряда. Миссия: наладить связь с удалённым сервером и получить важную информацию. Без правильной работы с API ты не справишься!

🚩 **Миссия 1: Кто такой API?**

Задание: Тебе нужно передать данные на сервер. Как ты это сделаешь?

Вопрос: Что такое API простыми словами?

Ответ:

- Это программа для взлома паролей
- **Это способ, по которому программы общаются между собой**
- Это тип памяти компьютера
- Это антивирус

🔔 **Шаг 03:**

🚩 **Миссия 2: API в реальной жизни**

Задание: Ты открываешь приложение погоды. Как оно получает данные?

Вопрос: Где мы встречаем API?

Ответ:

- Только в банковских приложениях

- Только в играх
- **В приложениях, которые получают данные с сервера, например, погода, карты, YouTube**
- **API** — это только для программистов и в жизни не встречается

🔮 Шаг 04:

🚩 Миссия 3: Программа на Go

Задание: Ты получил задание написать программу на Go, которая берёт шутку с сервера. Что тебе нужно сделать?

Вопрос: Какой пакет в Go используется для работы с API?

Ответ:

- `fmt/api`
- `json/http`
- **`net/http`**
- `string/utlis`

🔮 Шаг 05:

🚩 Миссия 4: Работа с JSON

Задание: Ты получил ответ от сервера в формате JSON. Что дальше?

Вопрос: Как правильно «расшифровать» JSON в Go?

Ответ:

- **С помощью команды `json.NewDecoder().Decode(...)`**
- Нужно скопировать ответ и вставить в Excel
- JSON автоматически отображается в браузере
- JSON невозможно использовать в Go

🔮 Шаг 06:

🚩 Миссия 5: Мини-программа

Задание: Ты хочешь получить случайную шутку через API. Какой заголовок ты должен установить в запросе?

Вопрос: Что написать в Header?

Ответ:

- "Accept: pizza"
- "Content-Type: video/mp4"
- **"Accept: application/json"**
- "Authorization: secret"

Урок 2 - Получаем погоду с сервера

Шаг 02:

Квест: «Прогноз погоды своими руками»

Ты — маленький программист и хочешь узнать, брать ли завтра зонт или ехать на велике. Но вместо того, чтобы открывать сайт, ты будешь получать погоду прямо из интернета!

Миссия 1. Зачем тянуть погоду из интернета?

Вопрос: Почему интереснее написать программу для прогноза погоды, а не просто открыть сайт?

Ответ:

- Потому что сайты скучные и нельзя программировать
- **Чтобы учиться общаться с серверами и получать данные самостоятельно**
- Потому что интернет сам подскажет всё без твоего участия

Шаг 03:

Миссия 2. Как работает интернет с погодой?

Интернет — это множество компьютеров, которые обмениваются данными. Сервер хранит данные о погоде, а мы с программой — клиенты.

Вопрос: Что делает наша программа?

Ответ:

- **Отправляет запрос на сервер → получает ответ → показывает погоду**
- Рисует картинку дождя без данных
- Смотрит прогноз на бумажке

Шаг 04:

Миссия 4. Чтение данных с сервера

JSON — это формат, в котором сервер присылает данные. Программа разбирает JSON и показывает: город, температуру, влажность и описание погоды.

Вопрос: Что мы делаем с JSON в программе?

Ответ:

- **Превращаем его в переменные и красиво выводим**
- Пишем его на стену
- Переводим в HTML

Шаг 05:

Миссия 5. Что можно улучшить?

Можно:

Ответ:

- Вводить город с клавиатуры
- Красить текст в терминале
- Сохранять погоду в файл
- Сделать Telegram-бота

Вопрос: Что из этого точно сделает программу круче для пользователя?

- Ввод города с клавиатуры
- Красим текст в терминале
- **Всё вместе**

Урок 3 - Telegram-бот на Go

Шаг 02:

Квест: «Создай своего Telegram-бота»

Ты — начинающий программист и хочешь, чтобы твоя программа отвечала на сообщения в Telegram. Сегодня ты создашь своего первого бота на Go!

Миссия 1. Что такое Telegram-бот? 🤖

Вопрос: Что умеет Telegram-бот?

Ответ:

- **Он может отправлять сообщения, картинки, шутки и реагировать на команды**
- Он только показывает рекламу
- Он заменяет человека в школе

Шаг 03:

Миссия 2. Что нужно, чтобы начать? ✂

Для работы нам понадобится:

Ответ:

- Go на компьютере
- Telegram-аккаунт
- Кодовый редактор (например, VS Code)
- Интернет

Вопрос: Что из этого точно нужно для создания бота?

- Только Telegram-аккаунт
- **Go, Telegram-аккаунт, редактор и интернет**
- Только интернет

Шаг 04:

Миссия 3. Регистрация бота 📄

Чтобы бот работал, Telegram даёт специальный токен через @BotFather.

Вопрос: Зачем нужен токен?

Ответ:

- **Чтобы бот мог общаться с Telegram от твоего имени**
- Чтобы открыть секретный уровень игры
- Чтобы бот мог рисовать графики

Шаг 05:

Миссия 4. Подключаем библиотеку 📦

В Go есть библиотека `tgbotapi`, которая помогает программе общаться с Telegram.

Вопрос: Что делает эта библиотека?

Ответ:

- **Отправляет и получает сообщения от Telegram**
- Создает музыкальные файлы
- Рисует графики

🔮 **Шаг 06:**

Миссия 5. Основной код бота 🖥️

Бот слушает сообщения и отвечает на команды, например /start .

Вопрос: Что делает этот кусок кода?

```
if update.Message.Text == "/start" {  
    msg := tgbotapi.NewMessage(update.Message.Chat.ID, "Привет! Я твой первый бот на Go!")  
    bot.Send(msg)  
}
```

Ответ:

- **Бот отвечает «Привет!» на команду /start**
- Бот удаляет все сообщения
- Бот запускает компьютер

🔮 **Шаг 07:**

Миссия 6. Хранение токена 🗄️

Вопрос: Почему токен нельзя вставлять прямо в код?

Ответ:

- **Чтобы его не украли и бот оставался безопасным**
- Чтобы бот работал быстрее
- Чтобы Telegram показывал рекламу

🔮 **Шаг 08:**

Миссия 7. Что можно улучшить? ✨

Можно добавить:

Ответ:

- Игру с угадыванием числа
- Команду /weather для погоды
- Случайные шутки или картинки

Вопрос: Какое улучшение сделает бота веселее для друзей?

- **Добавить случайные шутки и стикеры**
- Менять токен
- Убрать команду /start

Урок 4 - Мини-проект бот для поиска картинок

Шаг 02:

Квест: «Создай Telegram-бота для картинок»

Ты — разработчик и хочешь, чтобы твой бот присылал картинки по команде. Давай проверим, как ты справишься!

Миссия 1. Установка Go

Вопрос 1: Зачем нам нужен Go?

Ответ:

- Чтобы писать код бота
- Чтобы зарегистрироваться на Unsplash
- Чтобы открыть Telegram

Шаг 03:

Миссия 2. Регистрация бота в Telegram

Вопрос: Что выдаёт @BotFather после регистрации бота?

Ответ:

- **Token** — секретный ключ для работы бота
- Смайлик для бота
- Картинку

Шаг 04:

Миссия 3. Регистрация в Unsplash

Вопрос: Для чего нужен Unsplash API?

Ответ:

- Чтобы бот мог искать и присылать картинки
- Чтобы бот мог отправлять сообщения
- Чтобы бот играл музыку

Шаг 05:

Миссия 4. Подключение библиотеки Telegram API

Вопрос: Что делает библиотека tgbotapi ?

Ответ:

- Позволяет боту отправлять и получать сообщения и картинки
- Создаёт новые картинки
- Меняет токен

Шаг 06:

Миссия 5. Основной код бота

Бот слушает команды и ищет картинки по запросу.

Вопрос: Что делает этот кусок кода?

```
url := fmt.Sprintf("https://api.unsplash.com/search/photos?query=%s&client_id=%s", query, unsplashh  
resp, err := http.Get(url
```

Ответ:

- **Отправляет запрос к API Unsplash, чтобы найти картинки**
- Отправляет сообщение в Telegram
- Проверяет токен бота

🔮 **Шаг 07:**

Миссия 6. Переменные окружения 🗝️

Вопрос: Почему токены и ключи нельзя писать прямо в код?

Ответ:

- **Чтобы никто не украл ключи и бот оставался безопасным**
- Чтобы бот работал быстрее
- Чтобы Telegram прислал смайлик

🔮 **Шаг 08:**

Миссия 7. Запуск и проверка 🚀

Вопрос: Как проверить, что бот работает?

Ответ:

- **Написать боту слово, например cat или mountains, и получить картинку**
- Перезапустить компьютер
- Написать команду /start

🔮 **Шаг 09:**

Миссия 8. Идеи для улучшения ✨

Вопрос: Что можно добавить в бота, чтобы он был ещё круче?

Ответ:

- **Отправку нескольких картинок и команду /help**
- Менять токен каждый день
- Удалять все сообщения

📖 **Урок 5 - Викторина интернет и Go**

🔮 **Шаг 02:**

Что такое API?

Ответ:

- Программа для рисования
- **Интерфейс для общения между программами**
- Тип файла с музыкой
- Веб-браузер

🔮 **Шаг 03:**

Что значит «отправить запрос на сервер»?

Ответ:

- **Попросить сервер выполнить действие или дать данные**
- Скачать файл с компьютера

- Установить игру
- Открыть сайт в браузере

🔗 Шаг 04:

Какой протокол чаще всего используется для общения с API в интернете?

Ответ:

- FTP
- **HTTP**
- SMTP
- POP3

🔗 Шаг 05:

Что такое JSON?

Ответ:

- Язык программирования
- **Формат для обмена данными в интернете**
- Мобильное приложение
- Видео формат

🔗 Шаг 06:

Какой пакет в Go обычно используют для отправки HTTP-запросов?

Ответ:

- fmt
- **net/http**
- os
- math

🔗 Шаг 07:

Как в Go получить тело ответа от HTTP-запроса?

Ответ:

- **Использовать resp.Body и читать данные**
- Использовать fmt.Println
- Использовать os.Open
- Использовать json.Marshal

🔗 Шаг 08:

Что такое Telegram-бот?

Ответ:

- Робот для уборки
- **Программа, которая общается с пользователями в Telegram**
- Мобильный телефон
- Видеоигра

🔗 Шаг 09:

Для чего нужна библиотека `tgbotapi` в Go?

Ответ:

- Для работы с файлами
- **Для создания Telegram-ботов**
- Для рисования графиков
- Для отправки email

🔗 Шаг 10:

Что должен делать простой Telegram-бот?

Ответ:

- Писать письма
- **Отвечать на сообщения пользователей**
- Загружать видео
- Рисовать картинки

🔗 Шаг 11:

Что такое HTTP-запрос с методом POST?

Ответ:

- Запрос на получение данных
- **Запрос на отправку данных на сервер**
- Запрос на удаление файлов
- Запрос на просмотр видео

🔗 Шаг 12:

Что такое «парсинг JSON»?

Ответ:

- **Превращение JSON в структуру данных программы**
- Создание JSON файла
- Отправка JSON на сервер
- Сжатие файла

🔗 Шаг 13:

Что такое API ключ?

Ответ:

- **Пароль для доступа к API**
- Программа для работы с API
- Файл с данными
- Режим игры

🔗 Шаг 14:

Какую роль играет интернет в создании ботов?

Ответ:

- **Позволяет ботам общаться с серверами и получать данные**
- Делает ботов умными
- Хранит ботов на сервере
- Ускоряет работу компьютера

🔗 Шаг 15:

Что такое «мини-проект»?

Ответ:

- **Небольшая учебная программа**
- Компьютерная игра
- Большой сайт
- Приложение для рисования

🔗 Шаг 16:

Как бот может искать картинки?

Ответ:

- **Отправляя запросы к специальному API, который ищет изображения**
- Скачивая файлы из интернета без запросов
- Рисуя картинки сам
- Загружая картинки с компьютера пользователя

📖 Урок 6 - Задачи для закрепления

📖 Урок 7 - Викторина

🔗 Шаг 02:

Вопрос 1

Какой пакет в Go используется для отправки HTTP-запросов?

Подсказка: Он входит в стандартную библиотеку и часто используется для API.

Ответ:

- **net/http**
- os
- fmt
- io

🔗 Шаг 03:

Вопрос 2 Расположите шаги для выполнения GET-запроса в Go:

1. Создать запрос с помощью http.Get
2. Обработать ошибку
3. Прочитать тело ответа
4. Закрыть тело ответа

Подсказка: Сначала делаем запрос, затем читаем ответ и закрываем.

Ответ:

- 1. Создать запрос с помощью `http.Get`
- 2. Обработать ошибку
- 3. Прочитать тело ответа
- 4. Закрыть тело ответа

 Шаг 04:

Вопрос 3

Сопоставьте методы пакета `net/http` с их назначением.

Подсказка: Каждый метод делает своё дело при работе с HTTP.

Ответ:

- **`http.Get`** — Отправляет GET-запрос
- **`http.Post`** — Отправляет POST-запрос
- **`http.ListenAndServe`** — Запускает HTTP-сервер
- **`http.HandleFunc`** — Регистрирует обработчик пути

 Шаг 05:

Вопрос 4

Что делает функция ``http.ListenAndServe``?

Подсказка: Она запускает веб-сервер и слушает подключения.

Ответ:

- **Запускает HTTP-сервер на указанном порту**
- Отправляет GET-запрос
- Читает тело ответа
- Закрывает соединение клиента

 Шаг 06:

Вопрос 5 Расположите шаги создания простого веб-сервера в Go:

1. Создать функцию-обработчик
2. Зарегистрировать обработчик с `http.HandleFunc`
3. Запустить сервер с `http.ListenAndServe`
4. Обрабатывать запросы внутри функции

Подсказка: Сначала создаём функцию, потом регистрируем и запускаем сервер.

Ответ:

- 1. Создать функцию-обработчик
- 2. Зарегистрировать обработчик с `http.HandleFunc`
- 3. Запустить сервер с `http.ListenAndServe`
- 4. Обрабатывать запросы внутри функции

 Шаг 07:

Вопрос 6

Сопоставьте статус-коды HTTP с их значением.

Подсказка: Запомните основные коды: успешный, не найден, ошибка сервера.

Ответ:

- **200** — OK — запрос успешен
- **404** — Not Found — страница не найдена
- **500** — Internal Server Error — ошибка сервера
- **301** — Moved Permanently — перенаправление

🔗 Шаг 08:

Вопрос 7

Что важно делать после выполнения HTTP-запроса в Go?

Подсказка: Не забывайте закрывать ресурсы.

Ответ:

- **Закрыть тело ответа с `defer resp.Body.Close()`**
- Ничего не делать
- Закрыть сервер
- Повторно отправить запрос

🔗 Шаг 09:

Вопрос 8 Расположите шаги для работы с JSON API в Go:

1. Отправить HTTP-запрос
2. Прочитать тело ответа
3. Декодировать JSON в структуру
4. Обработать данные в коде

Подсказка: Сначала получаем данные, потом декодируем и используем.

Ответ:

- 1. Отправить HTTP-запрос
- 2. Прочитать тело ответа
- 3. Декодировать JSON в структуру
- 4. Обработать данные в коде

🔗 Шаг 10:

Вопрос 9

Сопоставьте форматы данных с типичным использованием в вебе:

Подсказка: JSON и XML часто используются для API, HTML для веб-страниц.

Ответ:

- **JSON** — Обмен данными между клиентом и сервером
- **XML** — Данные с вложенной структурой
- **HTML** — Веб-страницы
- **TXT** — Простой текстовый контент

🔗 Шаг 11:

Вопрос 10

Что важно учитывать при создании веб-сервиса в Go для большого количества пользователей?

Подсказка: Подумайте про нагрузку и параллельность.

Ответ:

- **Использовать горутины для обработки запросов параллельно**
- Обрабатывать все запросы последовательно
- Игнорировать ошибки при запросах
- Использовать глобальные переменные для всех данных

Модуль 5 - Модуль 5. Продвинутые возможности Go

Урок 1 - Пакеты делим проект на части

Шаг 02:

Квест: «Строй свой Go-город с пакетами»

Ты — архитектор кода, и твоя задача — превратить хаос в аккуратный город.

Миссия 1. Понимаем пакеты

Вопрос: Что такое пакет в Go?

Ответ:

- **Набор файлов с кодом, лежащих в одной папке и с одинаковым package в начале**
- Файл с картинками
- Команда для запуска бота

Шаг 03:

Миссия 2. Хаос без пакетов

Сценарий: У тебя есть игра с игроком и монстрами, и весь код лежит в main.go .

Вопрос: Какая главная проблема такого подхода?

Ответ:

- **Чем больше функций и структур, тем сложнее найти нужный кусок кода**
- Go не умеет запускать программы
- Файлы исчезают

Шаг 04:

Миссия. Красиво с пакетами

Теперь мы делим проект на папки player/ и monster/ .

Вопрос: Что делает пакет player ?

Ответ:

- **Хранит структуру и функции для игрока**
- Хранит картинки монстров
- Отправляет сообщения в Telegram

Шаг 05:

Миссия 4. Главный файл main.go

Вопрос: Почему структуры с заглавной буквы можно использовать в других пакетах?

Ответ:

- Потому что имена с заглавной буквы экспортируются и становятся видимыми вне пакета
- Потому что Go не любит маленькие буквы
- Потому что main.go это магический файл

Шаг 06:

Миссия 5. Запуск проекта

Вопрос: Что делает команда `go mod init game` ?

Ответ:

- Создает файл `go.mod`, чтобы Go понимал структуру проекта и мог правильно находить пакеты
- Запускает игру
- Создает нового игрока

Шаг 07:

Миссия 6. Практика на будущее

Вопрос: Когда стоит задуматься о пакетах?

Ответ:

- Когда проект больше 200–300 строк кода
- Когда проект меньше 50 строк
- Когда Go ругается

Урок 2 - Обработка ошибок без сбоев

Шаг 02:

Квест: «Робот-помощник и ошибки»

Ты — инженер по роботам-программистам. Твой робот выполняет задачи, но иногда сталкивается с препятствиями. Твоя цель — научить его не падать, а сообщать о проблемах .

Миссия 1. Понимаем ошибки

Вопрос 1: Что такое ошибка в Go?

Ответ:

- Страшный сбой, который ломает программу
- **Значение, которое сообщает: «Что-то пошло не так»**
- Новая переменная, которая создает файл

Шаг 03:

Миссия 2. Базовый шаблон обработки ошибок

Вопрос: Как правильно проверять ошибки в Go?

Ответ:

- После операции, которая может их вызвать: `результат, err := что_то_делаем() if err != nil { // обрабатываем ошибку }`
- Никогда не проверять
- олько через `panic()`

🔧 Шаг 04:

Миссия 3. Создаём свои ошибки ⚠️

Сценарий: Робот делит чай на чашки, но деление на ноль недопустимо.

Вопрос: Как сообщить о проблеме?

Ответ:

- Возвращать 0, `errors.New("деление на ноль запрещено")`
- Игнорировать
- Закрыть программу через `os.Exit(1)`

🔧 Шаг 05:

Миссия 4. Оборачиваем ошибки с контекстом 🔍

Вопрос: Зачем использовать `%w` в `fmt.Errorf` ?

Ответ:

- Чтобы добавить контекст к ошибке и потом её «развернуть»
- Чтобы Go выдавал больше паники
- Чтобы файл сам создавался

🔧 Шаг 06:

Миссия 5. Кроссплатформенные ошибки 🌐

Вопрос: Как сделать путь к файлу, чтобы программа работала и на Windows, и на Linux?

Ответ:

- `filepath.Join("data", "info.txt")`
- `"C:\\data\\info.txt"` только для Windows
- `"/home/user/data/file.txt"` только для Linux

🔧 Шаг 07:

Миссия 6. Когда ошибку можно игнорировать 🙶

Вопрос: Можно ли в Go игнорировать ошибки?

Ответ:

- Да, но только явно: `_ , _ = fmt.Println("Привет!")`
- Никогда
- Всегда, Go сам разберётся

🔧 Шаг 08:

Миссия 7. Мини-проект: Список друзей 📄

Сценарий: Робот читает список друзей из файла. Если файла нет, программа не должна падать, а показывать «Нет друзей в списке».

Вопрос: Какой подход верный?

Ответ:

- Проверяем `err` после `os.ReadFile` и подставляем значение по умолчанию
- Игнорируем `err` и надеемся на лучшее
- Закрываем программу через `panic()`

Урок 3 - Goroutines и каналы параллельность

Шаг 02:

Квест: «Кухня параллельных поваров»

Ты — шеф-программист на огромной кухне. У тебя есть много поваров (goroutines), которые готовят одновременно. Твоя цель — научить их работать параллельно и безопасно передавать блюда (данные) .

Миссия 1. Понимаем goroutines ✂

Вопрос: Что такое goroutine?

Ответ:

- Большой поток выполнения, как автобус
- **Лёгкий поток выполнения, как велосипед, который работает параллельно**
- Файл с кодом Go

Шаг 03:

Миссия 2. Каналы как «лента передачи» 📄

Вопрос: Для чего нужен канал (channel) в Go?

Ответ:

- **Чтобы goroutines могли безопасно обмениваться данными**
- Чтобы хранить переменные в памяти
- Чтобы останавливать программу

Шаг 04:

Миссия 3. Отправка и получение данных ⚡

Сценарий: Алиса готовит суп, Боб жарит котлеты. Мы хотим получить сообщение, когда блюдо готово.

Вопрос: Как правильно получить сообщение из канала?

Ответ:

- `msg := <-ch`
- `msg := ch.send()`
- `msg := ch.push()`

Шаг 05:

Миссия 5. Мини-проект: секундомер ⌚

Сценарий: Один поток считает секунды, другой печатает забавные сообщения.

Вопрос: Как правильно использовать канал для передачи секунд и завершения программы?

Ответ:

- **Отправлять значения в канал и закрывать его через `close(ch)`**
- Игнорировать канал, просто печатать через `fmt.Println`
- Использовать глобальные переменные вместо каналов

Шаг 06:

Миссия 6. Советы новичку 🧐

Вопрос: Как избежать проблем с goroutines и каналами?

Ответ:

- **Не запускать тысячи ненужных goroutines и закрывать каналы, когда они не нужны**
- Никогда не закрывать каналы
- Запускать всё в бесконечном цикле, чтобы было быстрее

Урок 4 - Мини-проект чат в терминале

 Шаг 02:

Квест: «Создай свой терминальный чат»

Ты — разработчик, который хочет переписываться с друзьями прямо в терминале. Но сначала тебе нужно понять, как устроен чат и как работают goroutines и сетевые соединения.

Миссия 1: Сетевые соединения

Вопрос: Что делает функция `net.Listen("tcp", ":8080")` в сервере?

Ответ:

- Подключает клиента к серверу
- **Запускает сервер и слушает порт 8080 для входящих подключений**
- Отправляет сообщения всем клиентам

 Шаг 03:

Миссия 2: Goroutines для параллельной работы ⚡

Сценарий: Клиент одновременно отправляет сообщения и получает новые от других клиентов.

Вопрос: Почему приём сообщений выполняется в отдельной goroutine?

Ответ:

- **Чтобы сообщения приходили одновременно с отправкой, без зависаний**
- Чтобы код работал медленнее
- Чтобы сервер не принимал новые подключения

 Шаг 04:

Миссия 3: Работа с клиентами на сервере 🤖

Вопрос: Как сервер пересылает сообщение всем клиентам кроме отправителя?

Ответ:

- **Через цикл `for _, client := range clients { if client != sender {...} }`**
- Через глобальную переменную `message`
- Через `fmt.Println()` в `main`

 Шаг 05:

Миссия 4: Буфер и сканер 📄

Вопрос: Что делает `bufio.NewScanner(conn)` в сервере и клиенте?

Ответ:

- **Позволяет читать сообщения построчно из соединения**
- Запускает новый goroutine
- Закрывает соединение

🔧 Шаг 06:

Миссия 5: Завершение работы клиента 🎯

Вопрос: Как клиент выходит из чата?

Ответ:

- **Вводит exit и прерывает цикл отправки сообщений**
- Закрывает сервер
- Отключает интернет
- 5002

🔧 Шаг 07:

Миссия 6: Запуск на разных ОС 🖥️

Вопрос: Что нужно сделать, чтобы сервер и клиент работали на Linux и Windows?

Ответ:

- **Скомпилировать с помощью go build для каждого файла и запускать соответствующие исполняемые файлы**
- Менять весь код под каждую ОС
- Использовать браузер вместо терминала

📖 Урок 5 - Викторина. Продвинутые возможности Go

🔧 Шаг 02:

Что такое пакет (package) в Go?

Ответ:

- Отдельный файл с кодом
- **Группа связанных файлов и функций для организации программы**
- Тип переменной
- Функция, которая выполняется первой

🔧 Шаг 03:

Как в Go обозначается пакет в файле?

Ответ:

- import
- func
- **package**
- var

🔧 Шаг 04:

Как подключить внешний пакет в Go?

Ответ:

- **import "имя_пакета"**
- include "имя_пакета"
- using "имя_пакета"
- require "имя_пакета"

🔗 Шаг 05:

Почему важно разбивать проект на пакеты?

Ответ:

- Для ускорения компиляции
- **Чтобы легче было организовывать и переиспользовать код**
- Чтобы работать только с глобальными переменными
- Для уменьшения размера кода

🔗 Шаг 06:

Какой тип значения возвращают функции в Go для обработки ошибок?

Ответ:

- bool
- string
- **error**

🔗 Шаг 07:

Что делает функция `panic()` ?

Ответ:

- Запускает функцию заново
- **Завершает программу с сообщением об ошибке**
- Обработывает ошибку и продолжает работу
- Логирует предупреждение

🔗 Шаг 08:

Как правильно проверить ошибку после вызова функции?

Ответ:

- `if err == nil { ... }`
- **`if err != nil { ... }`**
- `if error { ... }`
- `if error == false { ... }`

🔗 Шаг 09:

Что такое `goroutine` в Go?

Ответ:

- Новый тип данных
- **Лёгкий поток выполнения для параллельной работы**
- Функция для работы с сетью
- Ошибка времени выполнения

🔗 Шаг 10:

Как запустить функцию в отдельной `goroutine`?

Ответ:

- `run функция()`

- start функция()
- **go функция()**
- thread функция()

Шаг 11:

Что такое канал (channel) в Go?

Ответ:

- Структура для хранения данных
- **Механизм для передачи данных между goroutine**
- Тип функции
- Ссылка на файл

Шаг 12:

Как объявить канал для передачи чисел?

Ответ:

- **var ch chan int**
- var ch int chan
- var ch channel int
- var ch int

Урок 6 - Викторина

Шаг 02:

Вопрос 1

Что делает ключевое слово `defer` в Go?

Подсказка: Оно откладывает выполнение до конца функции.

Ответ:

- **Откладывает выполнение функции до завершения текущей функции**
- Прерывает выполнение функции
- Создает новую горутину
- Вызывает функцию сразу же

Шаг 03:

Вопрос 2 Расположите шаги работы с каналами Go для передачи данных между горутинами:

1. Создать канал
2. Отправить данные в канал
3. Прочитать данные из канала
4. Закрыть канал

Подсказка: Сначала создаем канал, потом используем его, и в конце закрываем.

Ответ:

- 1. Создать канал
- 2. Отправить данные в канал
- 3. Прочитать данные из канала
- 4. Закрыть канал

🔗 Шаг 04:

Вопрос 3

Сопоставьте типы каналов Go с их назначением.

Подсказка: Каналы могут быть только для чтения, только для записи или для обоих.

Ответ:

- **chan T** — Двухнаправленный канал, можно читать и писать
- **chan<- T** — Канал только для отправки данных
- **<-chan T** — Канал только для получения данных

🔗 Шаг 05:

Вопрос 4

Что делает функция ``select`` в Go при работе с каналами?

Подсказка: Она позволяет выбирать один из нескольких каналов для чтения или записи.

Ответ:

- **Позволяет работать с несколькими каналами одновременно**
- Закрывает канал
- Создает новый канал
- Запускает новую горутину

🔗 Шаг 06:

Вопрос 5 Расположите шаги работы с мьютексами для синхронизации горутин:

1. Создать объект мьютекса
2. Вызвать Lock перед доступом к разделяемым данным
3. Вызвать Unlock после завершения работы с данными
4. Горутины безопасно используют данные

Подсказка: Сначала создаем мьютекс, блокируем доступ, работаем с данными и разблокируем.

Ответ:

- 1. Создать объект мьютекса
- 2. Вызвать Lock перед доступом к разделяемым данным
- 3. Вызвать Unlock после завершения работы с данными
- 4. Горутины безопасно используют данные

🔗 Шаг 07:

Вопрос 6

Сопоставьте структуры данных Go с их назначением:

Подсказка: Каждая структура имеет свои сильные стороны.

Ответ:

- **map** — Словарь для хранения пар ключ-значение
- **slice** — Динамический массив
- **array** — Фиксированный массив элементов
- **struct** — Объединяет разные поля в одну сущность

🔮 Шаг 08:

Вопрос 7

Что важно учитывать при использовании горутин в Go?

Подсказка: Горутин много, но данные должны быть защищены.

Ответ:

- **Синхронизация доступа к разделяемым данным**
- Создавать горутин без контроля
- Использовать только один канал
- Игнорировать ошибки горутин

🔮 Шаг 09:

Вопрос 8 Расположите шаги обработки ошибок с помощью ``recover`` и ``panic``:

1. Вызвать `panic` при критической ошибке
2. Использовать `defer` с функцией `recover` для перехвата паники
3. Обработать ошибку или логировать
4. Продолжить выполнение программы безопасно

Подсказка: `Panic` вызывает остановку, `recover` позволяет спасти программу.

Ответ:

- 1. Вызвать `panic` при критической ошибке
- 2. Использовать `defer` с функцией `recover` для перехвата паники
- 3. Обработать ошибку или логировать
- 4. Продолжить выполнение программы безопасно

🔮 Шаг 10:

Вопрос 9

Сопоставьте функции пакета ``time`` с их назначением:

Подсказка: Время в Go полезно для таймеров, задержек и получения текущего момента.

Ответ:

- **`time.Now()`** — Получить текущую дату и время
- **`time.Sleep`** — Приостановить выполнение на заданное время
- **`time.Tick`** — Создает канал, который передает время через интервалы
- **`time.After`** — Возвращает канал, который посылает сигнал после заданного времени

🔮 Шаг 11:

Вопрос 10

Что важно учитывать при работе с пакетами и модулями Go для больших проектов?

Подсказка: Организация кода помогает поддерживать проект чистым и понятным.

Ответ:

- **Создавать модули для логической группировки функций и структур**
- Ставить все функции в один файл `main.go`
- Игнорировать зависимости
- Использовать глобальные переменные для всего

Модуль 6 - Модуль 6. Финальный проект — первый стартап

Урок 1 - Выбор идеи бот, сайт или сервис

Шаг 02:

Квест: «Выбери свой проект»

Ты — разработчик, который уже умеет писать на Go. Перед тобой лежит выбор: бот, сайт или сервис. Каждый путь — это своё приключение. Твоя задача — пройти шаги к правильной идее.

Миссия 1: Понимание своих интересов ☺

Вопрос: Что нужно сделать в первую очередь, чтобы выбрать проект?

Ответ:

- Выбрать самое сложное, что можно придумать
- **Записать 3-5 своих интересов (игры, музыка, спорт, мемы, учёба...)**
- Сразу начать кодить

Шаг 03:

Миссия 2: Решаем проблему ✂

Сценарий: Ты записал свои интересы. Теперь нужно придумать, какую проблему можно решить в этих темах.

Вопрос: Как лучше выбрать проект?

Ответ:

- **Сделать что-то полезное и реально выполнимое за 1-2 месяца**
- Сделать «супербота, который знает всё», даже если не хватает навыков
- Начать с самого сложного сайта с базой данных и машинным обучением

Шаг 04:

Миссия 3: Выбор направления ☆

Выбери одно из трёх направлений:

Бот

Ответ:

- Живёт в мессенджере (Telegram, VK, Discord).
- Быстро сделать рабочий прототип.
- Можно подключать API (погода, картинки, игры).

Примеры: бот-помощник по домашке, бот с мемами, бот-напоминалка.

Сайт

- Показывает проект всему миру через браузер.
- Осваиваешь веб-разработку на Go.
- Можно экспериментировать с дизайном.

Примеры: личный блог, онлайн-счётчик, мини-игра.

Сервис

- Работает «в фоне», выполняет задачи автоматически.

- Полезно для личного использования.
- Можно работать с файлами, API, базами данных.

Примеры: сервис новостей, музыкальный плеер, генератор паролей.

Вопрос: Какое направление выбрать?

- **То, что лично интересно и реально сделать**
- Самое сложное и «крутое»
- Любое, что понравилось другу

Шаг 05:

Миссия 4: Мини-чеклист перед стартом ✓

Вопрос: Какие пункты нужно проверить, прежде чем начинать проект?

Ответ:

- **Мне интересно этим заниматься**
- **Я смогу сделать базовую версию за 1-2 недели**
- **Я смогу показать проект друзьям**
- **Есть идеи, что добавить в будущем**

Шаг 06:

Миссия 5: Первые шаги к реализации ✍️

Вопрос: Что делать после выбора идеи?

Ответ:

- **Разбить проект на маленькие задачи и решать по одной за раз**
- Сразу писать весь код без плана
- Искать сложные библиотеки для усложнения

Урок 2 - Архитектура проекта

Шаг 02:

Квест: «Построй свой проект»

Ты решил создать своё приложение на Go. Перед тобой три этапа: понять архитектуру, спланировать структуру и правильно организовать код.

Миссия 1: Понимаем архитектуру 🏠

Вопрос: Что такое архитектура проекта?

Ответ:

- Способ красиво оформить код, добавить шрифты и цвета
- **План, как устроены и взаимодействуют части программы**
- Выбор языка программирования

Шаг 03:

Миссия 2: Разделяем обязанности 🤖

Представь, что ты делаешь Telegram-бота.

Вопрос: Что должно быть в модуле handlers ?

Ответ:

- Получение данных с сервера погоды и отправка сообщений
- **Логика реакции на команды пользователя**
- Настройки токенов и путей

🔮 **Шаг 04:**

Миссия 3: Минимизируем зависимости 🔗

Вопрос: Почему важно, чтобы модули зависели друг от друга минимально?

Ответ:

- **Чтобы менять код одного модуля не ломало остальные**
- Чтобы проект был сложнее
- Чтобы использовать больше памяти

🔮 **Шаг 05:**

Миссия 4: Читаем как книгу 📖

Открыл проект — и сразу понятно, где логика бота, где настройки, а где работа с базой.

Вопрос: Что помогает достичь этого эффекта?

Ответ:

- **Разделение кода на пакеты**
- Использование длинных функций без комментариев
- Всё в одном файле

🔮 **Шаг 06:**

Миссия 5: Создаём свой ToDo-проект ✔

Шаг 1: Выбираем структуру:

```
todo/  
├── main.go  
├── tasks/  
│   ├── add.go  
│   └── list.go  
└── utils/  
    └── storage.go
```

Вопрос: Что должно быть в main.go ?

Ответ:

- Реализовывать все функции добавления и отображения задач
- **Запускать программу и подключать модули**
- Сохранять задачи в файлы

🔮 **Шаг 07:**

Миссия 6: Кроссплатформа и пути 🌐

Вопрос: Как лучше писать пути в Go, чтобы проект работал на Windows и Linux?

Ответ:

- Использовать \ для Windows и / для Linux

- Всегда использовать /
- Писать пути с пробелами

Урок 3 - Реализация от идеи к коду

Шаг 02:

Квест: «От идеи к коду»

Ты уже построил план своего проекта (архитектуру). Теперь пора оживить его с помощью кода.

Миссия 1: Создаём структуру проекта

Вопрос: Какая папка должна содержать главный файл запуска приложения?

Ответ:

- internal/services
- **cmd/myapp**
- pkg

Шаг 03:

Миссия 2: Обработчики команд

Ты создаёшь обработчик SayHello .

Вопрос: Где должен лежать файл hello.go ?

Ответ:

- **internal/handlers**
- internal/services
- pkg

Шаг 04:

Миссия 3: Бизнес-логика

Функция GetGreeting формирует приветствие.

Вопрос: Где её правильное место?

Ответ:

- **internal/services/greeting.go**
- internal/handlers/hello.go
- main.go
- 5002

Шаг 05:

Миссия 4: Используем memory.go

Ты хочешь сохранять последнее имя пользователя.

Вопрос: Почему memory.go пока не влияет на вывод?

Ответ:

- Потому что Go игнорирует этот файл
- **Потому что его функции не вызываются в других модулях**
- Потому что память пустая

💡 Шаг 06:

Миссия 5: Подключаем `utils.go` ↩

Ты написал функцию `Capitalize` .

Вопрос: Как её правильно использовать?

Ответ:

- Вызывать в `services` для форматирования имени
- Писать заново в каждом файле
- Оставить неиспользованной

💡 Шаг 07:

Миссия 6: Запуск проекта 🚀

Вопрос: Как запустить проект в терминале?

Ответ:

- `go build main.go`
- **`go run ./cmd/myapp`**
- `go start main.go`

💡 Шаг 08:

Миссия 7: Проверяем вывод ☹

После запуска проекта ты должен увидеть примерно:

Приложение запускается...

Привет, Школьник-разработчик! Добро пожаловать в мир Go!

Последний пользователь в памяти: Школьник-разработчик

Вопрос: Почему отображается последний пользователь?

Ответ:

- Потому что `SayHello` вызывает `services.GetLastUser()`
- Потому что Go сам запоминает переменные
- Потому что это тестовый вывод

📖 Урок 4 - Презентация проекта

💡 Шаг 02:

🏆 Квест: «Мой путь в Go»

Ты рассказываешь о том, чему учишься и почему это важно. Каждое задание — шаг к пониманию Go и твоих навыков.

Миссия 1: Что такое Go? 🤖

Вопрос: Какое утверждение верно о языке Go?

Ответ:

- Это язык от Google, который быстрый и современный
- Это устаревший язык, используемый только для игр
- Go — это только для веб-дизайна

🔮 Шаг 03:

Миссия 2: Почему Go? ⚡

Вопрос: Что делает Go удобным для изучения новичками?

Ответ:

- Код прост и читаем
- Код сложный и запутанный
- Go не имеет документации

🔮 Шаг 04:

Миссия 3: Кто использует Go? 🌐

Вопрос: Какие компании используют Go?

Ответ:

- Google, Uber, Twitch, Dropbox
- Только маленькие стартапы
- Компании используют только Python

🔮 Шаг 05:

Миссия 4: Что я уже умею делать? ✂

Вопрос: Какие навыки ты получил благодаря курсу?

Ответ:

- Создавать программы, мини-игры, чат-ботов
- Только читать чужой код
- Писать программы, которые не запускаются

🔮 Шаг 06:

Миссия 5: Как Go помогает в жизни и школе? 🧠

Вопрос: Что развивает изучение Go?

Ответ:

- Логику, умение решать задачи, творческое мышление
- Только память на числа
- Навык рисования

🔮 Шаг 07:

Миссия 6: Мои достижения 🏆

Вопрос: Что ты можешь показать прямо сейчас?

Ответ:

- Первые программы, мини-игры, проекты с файлами и интернетом
- Только теорию и книги
- Готовые программы других людей

🔮 Шаг 08:

Миссия 7: Как люди могут помочь тебе? 🤝

Вопрос: Как поддержка других помогает в обучении?

Ответ:

- **Интересоваться успехами, создавать комфортные условия**
- Не спрашивать и не показывать ничего
- Делать проекты за тебя

🔮 **Шаг 09:**

Финальная миссия: Почему это круто? ☆

Вопрос: Почему изучение Go полезно?

Ответ:

- **Открывает возможности для профессий, тренирует мозг и развивает креативность**
- Только для развлечения без пользы
- Только для работы в школе

📖 **Урок 5 - Публикация кода на GitHub**

🔮 **Шаг 02:**

Миссия 1: Что такое GitHub? 😊

Вопрос: Какое утверждение верно?

Ответ:

- **GitHub — это онлайн-сервис для хранения и совместной работы с кодом**
- **GitHub — это игра для программистов**
- GitHub нужен только для скачивания чужих программ

🔮 **Шаг 03:**

Миссия 2: Зачем выкладывать проекты на GitHub? 🌐

Вопрос: Почему важно публиковать свои проекты?

Ответ:

- **Создаешь портфолио и показываешь успехи другим**
- Чтобы только хранить код для себя
- Чтобы создавать вирусы

🔮 **Шаг 04:**

Миссия 3: Подготовка проекта ✂

Вопрос: Что нужно сделать перед публикацией проекта?

Ответ:

- **Проверить код, создать папку проекта и добавить README**
- Просто скопировать файлы на GitHub без проверки
- Писать код прямо на сайте GitHub

🔮 **Шаг 05:**

Миссия 4: Создание аккаунта 👤

Вопрос: Как зарегистрироваться на GitHub?

Ответ:

- Зайти на **github.com**, придумать уникальное имя и пароль, подтвердить почту
- Просто войти через Google без регистрации
- Невозможно зарегистрироваться бесплатно

Шаг 06:

Миссия 5: Git — твой помощник

Вопрос: Зачем нужен Git?

Ответ:

- Чтобы сохранять версии кода и загружать их в интернет
- Чтобы писать код быстрее
- Чтобы создавать картинки

Шаг 07:

Миссия 6: Создание репозитория

Вопрос: Что нужно сделать для создания репозитория на GitHub?

Ответ:

- Нажать **New repository**, придумать имя, добавить описание, создать README
- Просто скопировать папку с компьютера
- Открыть терминал и писать код без репозитория

Шаг 08:

Миссия 7: Загружаем проект

Вопрос: Какие команды Git выполняют загрузку проекта на GitHub?

Ответ:

- **git init, git add ., git commit -m "Первый коммит", git branch -M main, git remote add origin URL, git push -u origin main**
- python run.py
- npm install

Шаг 09:

Миссия 8: Проверяем результат

Вопрос: Как понять, что проект успешно загружен?

Ответ:

- Обновить страницу репозитория — файлы должны появиться
- Проверить на компьютере
- Ждать 24 часа

Шаг 10:

Миссия 9: Советы для новичков

Вопрос: Что помогает работать с проектами правильно?

Ответ:

- Часто делать коммиты, писать понятные комментарии, использовать **.gitignore**

- Никогда не делать коммиты
- Загружать только рабочие файлы без комментариев

🔮 Шаг 11:

Финальная миссия: Зачем нужен GitHub? ✨

Вопрос: Как еще GitHub помогает в обучении и карьере?

Ответ:

- Совместная работа, портфолио, участие в конкурсах, изучение чужого кода
- Только для хранения личного кода
- Чтобы играть в игры

📖 Урок 6 - Викторина

🔮 Шаг 02:

Вопрос 1

Какой тип проекта лучше всего начать для закрепления базовых навыков Go?

Подсказка: Выбирайте что-то простое, чтобы освоить синтаксис и структуры данных.

Ответ:

- Мини-сервис с выводом текста на веб-странице
- Полноценная социальная сеть
- Игровой движок уровня AAA
- Комплексная ERP-система

🔮 Шаг 03:

Вопрос 2 Расположите шаги создания веб-сервиса на Go в правильном порядке:

1. Создание структуры проекта и папок
2. Определение маршрутов и обработчиков
3. Тестирование работы сервиса локально
4. Запуск сервиса и проверка через браузер

🔮 Подсказка:

Сначала подготовка структуры, затем маршруты, потом тесты, и только в конце запуск.

Ответ:

- 1. Создание структуры проекта и папок
- 2. Определение маршрутов и обработчиков
- 3. Тестирование работы сервиса локально
- 4. Запуск сервиса и проверка через браузер

🔮 Шаг 04:

Вопрос 3

Какой метод лучше использовать для хранения данных пользователей в учебном проекте?

Подсказка: Для старта хватит чего-то простого, не обязательно база данных.

Ответ:

- Использовать встроенную структуру map в Go
- Сразу подключать полноценную SQL-базу

- Хранить данные в текстовых файлах на сервере без структуры
- Использовать внешнее облачное хранилище без API

🔗 Шаг 05:

Вопрос 4

Сопоставьте элементы Go-проекта с их назначением:

Элементы:

Ответ:

- main.go
- handlers.go
- routes.go
- templates/

Назначения:

1. Главный файл запуска приложения
2. Функции обработки запросов
3. Маршруты и связывание с обработчиками
4. HTML-шаблоны для отображения

🔗 Подсказка:

Главное разделение: код запуска, обработчики, маршруты и визуальная часть.

- **main.go** — Главный файл запуска приложения
- **handlers.go** — Функции обработки запросов
- **routes.go** — Маршруты и связывание с обработчиками
- **templates/** — HTML-шаблоны для отображения

🔗 Шаг 06:

Вопрос 5

Какой способ отладки кода наиболее эффективен для начинающего Go-разработчика?

Подсказка: Используйте встроенные средства и маленькие примеры кода.

Ответ:

- **Добавлять `fmt.Println` для проверки переменных**
- Использовать сложные внешние отладчики сразу
- Игнорировать ошибки и надеяться на удачу
- Удалять случайные строки кода

🔗 Шаг 07:

Вопрос 6 Расположите этапы создания бота на Go в правильном порядке:

1. Регистрация бота и получение токена
2. Подключение к API и создание обработчиков сообщений
3. Тестирование бота с простыми командами
4. Запуск и проверка работы в реальном чате

🔗 Подсказка:

Сначала регистрация, затем подключение, потом тесты, в конце - запуск.

Ответ:

- 1. Регистрация бота и получение токена
- 2. Подключение к API и создание обработчиков сообщений
- 3. Тестирование бота с простыми командами
- 4. Запуск и проверка работы в реальном чате

🔗 Шаг 08:

Вопрос 7

Какой подход к структуре кода помогает легко добавлять новые функции в проект на Go?

Подсказка: Используйте модульность и отделение логики друг от друга.

Ответ:

- **Разделять код на пакеты по функционалу**
- Собирать весь код в одном файле
- Использовать только глобальные переменные
- Игнорировать структуру и добавлять функции хаотично

🔗 Шаг 09:

Вопрос 8

Сопоставьте типы Go-проектов с примерами:

Ответ:

- Веб-сервис
 - Чат-бот
 - CLI-сервис
 - API-сервис
1. Онлайн-магазин
 2. Telegram-бот
 3. Утилита для обработки файлов
 4. Сервис для интеграции с мобильным приложением

🔗 Подсказка:

Соотнесите назначение проекта с типом сервиса.

- **Веб-сервис** — Онлайн-магазин
- **Чат-бот** — Telegram-бот
- **CLI-сервис** — Утилита для обработки файлов
- **API-сервис** — Сервис для интеграции с мобильным приложением

🔗 Шаг 10:

Вопрос 9

Что поможет сделать ваш проект на Go более удобным для других разработчиков?

Подсказка: Документация и чистый код важнее, чем сложные трюки.

Ответ:

- **Писать понятные функции и комментарии**
- Использовать только одну длинную функцию на всё
- Игнорировать ошибки и исключения
- Не тестировать код перед публикацией

Шаг 11:

Вопрос 10 Расположите шаги финального тестирования проекта на Go в правильном порядке:

1. Проверка всех функций вручную
2. Запуск автоматизированных тестов
3. Исправление найденных ошибок
4. Проверка совместимости с различными платформами

Подсказка:

Сначала ручная проверка, затем автоматические тесты, исправления и финальная совместимость.

Ответ:

- 1. Проверка всех функций вручную
- 2. Запуск автоматизированных тестов
- 3. Исправление найденных ошибок
- 4. Проверка совместимости с различными платформами

Модуль 7 - Модуль 7. Путь после курса

Урок 1 - Как школьнику найти подработку с Go

Шаг 02:

Квест: «Школьник + Go = первая подработка»

Ты уже умеешь писать программы на Go. Теперь пора узнать, как превратить знания в опыт и деньги! Каждое задание — шаг к первой подработке.

Миссия 1: Почему Go — крутой для подработки? 😊

Вопрос: Что делает Go отличным языком для начинающего разработчика?

Ответ:

- Язык сложный, поэтому все платят больше
- **Go простой, востребованный и подходит для удалённой работы**
- Go нужен только для игр

Шаг 03:

Миссия 2: Где искать задания? 🌐

Вопрос: Какие места лучше всего подходят для поиска подработки с Go?

Ответ:

- **Фриланс-платформы, соцсети, локальные знакомства, школьные проекты**
- Только друзья и родственники
- Игровые форумы

Шаг 04:

Миссия 3: Какие задачи подойдут новичку? ✂

Вопрос: Что может делать новичок на Go?

Ответ:

- Простейшие боты, автоматизация рутинных процессов, создание сайтов с простым функционалом, помощь в тестировании
- Создание сложных видеоигр с 3D графикой
- Управление космическими станциями

Шаг 05:

Миссия 4: Как подготовиться к подработке?

Вопрос: Какие шаги помогут подготовиться к первой задаче?

Ответ:

- Сделать портфолио, написать резюме, учиться общаться с заказчиками, начинать с маленьких задач
- Просто искать заказы без подготовки
- Писать код без комментариев и примеров

Шаг 06:

Миссия 5: Советы для успешной работы

Вопрос: Что помогает работать и зарабатывать правильно?

Ответ:

- Планировать время, тестировать код, просить обратную связь, быть честным, развивать навыки
- Игнорировать ошибки и сроки
- Брать все заказы подряд, не изучая задачу

Шаг 07:

Миссия 6: Пример выполнения задачи

Вопрос: Как правильно выполнить простую задачу, например, создать Telegram-бота?

Ответ:

- Зарегистрироваться на фриланс-платформе → найти задачу → написать заказчику и показать проекты → обсудить детали → сделать программу → получить оплату и отзыв
- Сделать программу сразу и ждать, что заказчик сам найдет тебя
- Просто скачать чужого бота и отправить

Шаг 08:

Миссия 7: Как подработка помогает будущему?

Вопрос: Какие преимущества даёт подработка с Go школьнику?

Ответ:

- Получение навыков, опыт работы с людьми, повышение шансов на университет или стажировку, ощущение независимости
- Только возможность тратить деньги
- Нет никакой пользы

Урок 2 - Где искать заказы фриланс, хакатоны, стартапы

Шаг 02:

Квест: «Твои первые проекты с Go»

Ты умеешь писать на Go и хочешь применить знания на практике. В этом квесте ты узнаешь, где искать заказы, как участвовать в стартапах и хакатонах, и как начать зарабатывать опыт.

Миссия 1: Фриланс — работа на заказ через интернет

Вопрос: Где можно найти первые фриланс-заказы на Go?

Ответ:

- **Upwork, Freelancer, Kwork, Work-Zilla, YouDo**
- Только друзья и родственники
- Форумы о музыке

Шаг 03:

Миссия 2: Стартапы — создаём новое вместе

Вопрос: Почему работать со стартапами полезно для новичка?

Ответ:

- **Можно участвовать в создании нового продукта, научиться быстро решать задачи, работать в команде**
- Стартапы платят всегда очень много, сразу миллионы
- **Стартапы** — это только игры и развлечения

Шаг 04:

Миссия 3: Хакатоны — соревнуйся и учись быстро

Вопрос: Что такое хакатон?

Ответ:

- **Мероприятие, где за 24-48 часов создают работающие проекты в командах**
- Долгий курс по Go
- Онлайн-магазин для программистов

Шаг 05:

Миссия 4: Как подготовиться к первому заказу или проекту

Вопрос: Какие шаги помогут успешно начать работу?

Ответ:

- **Сделать портфолио, учиться Go, практиковаться, планировать задачи, быть вежливым и ответственным**
- Игнорировать обучение и сразу брать самые сложные проекты
- Работать только над играми и мемами

Шаг 06:

Миссия 5: Итоговый выбор — с чего начать?

Вопрос: Какой путь выбрать новичку для первых проектов с Go?

Ответ:

- Начать с простого фриланса → попробовать стартапы → участвовать в хакатонах
- Сразу создать крупный стартап и ждать миллионов
- Только смотреть чужие проекты и ничего не делать

Урок 3 - Как учиться дальше и расти в IT

Шаг 02:

Квест: «Путь разработчика Go»

Ты уже освоил основы Go и сделал первые проекты. Теперь пора двигаться дальше, прокачивать навыки и становиться круче в IT.

Миссия 1: Определи цель обучения 🎯

Вопрос: Почему важно ставить цель при обучении?

Ответ:

- Чтобы понимать, что и зачем учить, и видеть результат
- Чтобы сразу стать миллиардером
- Чтобы меньше программировать и больше отдыхать

Шаг 03:

Миссия 2: Планирование времени ⌚

Вопрос: Как лучше организовать обучение?

Ответ:

- Учиться регулярно, даже по 30 минут в день, делать перерывы и записывать новые знания
- Учить всё за один день без отдыха
- Смотреть видео и ничего не пробовать писать

Шаг 04:

Миссия 3: Выбираем источники знаний 📖

Вопрос: Какие источники помогут развиваться как Go-разработчику?

Ответ:

- Книги, онлайн-курсы, практика, общение в сообществах
- Только мемы про программистов
- Ставить галочки в соцсетях без кодинга

Шаг 05:

Миссия 4: Развиваем продвинутые навыки Go ✂

Вопрос: Какие навыки стоит прокачивать после базового Go?

Ответ:

- Интерфейсы, структуры, goroutines, каналы, работа с файлами и сетью, тестирование
- Только игры и графику
- Учить другие языки, забывая Go

Шаг 06:

Миссия 5: Мягкие навыки

Вопрос: Какие «мягкие» навыки важны для программиста?

Ответ:

- **Коммуникация, тайм-менеджмент, критическое мышление, командная работа**
- Только быстрый набор кода
- Спать и не общаться с людьми

Шаг 07:

Миссия 6: Практика и проекты

Вопрос: Как лучше всего прокачивать навыки Go?

Ответ:

- **Делать мини-проекты, участвовать в хакатонах, решать задачи на LeetCode, помогать другим**
- Смотреть чужой код и не пробовать писать свой
- Играть в игры вместо программирования

Урок 4 - Личный бренд портфолио и резюме

Шаг 02:

Квест: «Создай свой личный бренд в IT»

Ты учишь Go, создаёшь проекты и хочешь, чтобы о тебе знали. Личный бренд помогает стать заметным и открывает двери к стажировкам, подработкам и работе мечты.

Миссия 1: Что такое личный бренд?

Вопрос: Что входит в личный бренд программиста?

Ответ:

- **Портфолио, резюме, профиль в LinkedIn**
- Только игры и мемы
- Картинки котиков на GitHub

Шаг 03:

Миссия 2: Создаем портфолио

Вопрос: Как лучше показать свои проекты?

Ответ:

- **Хранить код на GitHub и сделать сайт с описанием проектов**
- Ставить все проекты в папку на компьютере и никому не показывать
- Писать о проектах только в соцсетях

Шаг 04:

Миссия 3: Пишем резюме школьнику

Вопрос: Какие разделы должны быть в твоём резюме?

Ответ:

- **Контактные данные, цель, навыки, проекты, образование, дополнительно**

- Только имя и любимые игры
- Только длинное описание всей жизни

🔮 Шаг 06:

Миссия 4: Профиль в LinkedIn 🌐

Вопрос: Для чего нужен LinkedIn?

Ответ:

- Чтобы показывать себя в профессиональном мире, находить контакты и компании
- Чтобы писать посты о любимых мемах
- Чтобы только лайкать друзей

🔮 Шаг 07:

Миссия 5: Почему личный бренд важен? ✨

Вопрос: Что даёт хорошо сделанный личный бренд?

Ответ:

- Работодатели тебя находят, создаётся первое впечатление, доступ к контактам, уверенность и мотивация
- Только модные аватарки
- Ничего — лучше скрывать свои достижения

📖 Урок 5 - Секреты быстрых и качественных проектов

🔮 Шаг 02:

🏆 Квест: «Создай крутой проект быстро и без стресса»

Ты уже умеешь писать на Go, но хочешь делать проекты эффективно — быстро и качественно. Давай проверим, как ты применяешь секреты продуктивной работы!

Миссия 1: Планируем проект 📄

Вопрос: С чего лучше начать проект?

Ответ:

- Определить цель проекта, разбить задачу на шаги, нарисовать схему
- Сразу писать весь код без плана
- Смотреть чужие проекты и копировать

🔮 Шаг 03:

Миссия 2: Пишем понятный код 📖

Вопрос: Как сделать код проще и удобнее для чтения?

Ответ:

- Использовать понятные имена переменных, маленькие функции, комментировать сложные места
- Писать длинные функции с непонятными названиями
- Не комментировать и всё запихивать в одну функцию

🔮 Шаг 04:

Миссия 3: Функции, пакеты и повторное использование ↩️

Вопрос: Зачем делить проект на функции и пакеты?

Ответ:

- **Чтобы повторно использовать код и не менять весь проект при добавлении нового**
- Чтобы всё было в одном файле
- Чтобы код стал длиннее и сложнее

Шаг 05:

Миссия 4: Тестируем по частям

Вопрос: Когда лучше тестировать код?

Ответ:

- **Сразу после написания каждой части**
- Только когда весь проект готов
- Не тестировать и надеяться, что всё работает

Шаг 06:

Миссия 5: Git и контроль версий

Вопрос: Почему полезно использовать Git?

Ответ:

- **Можно вернуться к рабочему варианту кода и работать в команде**
- Чтобы хранить код в Google Docs
- Чтобы никто не видел твой проект

Шаг 07:

Миссия 6: Используем библиотеки и инструменты

Вопрос: Почему лучше использовать готовые библиотеки?

Ответ:

- **Экономишь время и делаешь проекты качественнее**
- Чтобы всё писать с нуля, даже если есть готовое
- Чтобы код был длиннее

Шаг 08:

Миссия 7: Работа с другими и помощь

Вопрос: Почему полезно работать с другими?

Ответ:

- **Легче найти лучшее решение и учиться у опытных**
- Чтобы показывать проекты друзьям без пользы
- Чтобы никто не вмешивался

Шаг 09:

Миссия 8: Перерывы и отдых

Вопрос: Зачем делать паузы?

Ответ:

- **Мозг работает лучше, идеи приходят быстрее, ошибки становятся заметнее**

- Чтобы тратить время зря
- Чтобы отложить проект на потом

Урок 6 - Викторина

Шаг 02:

Вопрос 1

Сопоставьте тип проекта с его описанием.

Подсказка: Подумайте, какие задачи решает каждый проект.

Ответ:

- **Чат-бот** — Программа, которая автоматически отвечает на сообщения пользователей.
- **Веб-магазин** — Сайт для продажи товаров с корзиной и оплатой.
- **API-сервис аналитики** — Сервис для сбора и анализа данных с внешних источников.
- **Консольное приложение для вычислений** — Программа, которая выполняет математические вычисления через командную строку.

Шаг 03:

Вопрос 2

Сопоставьте тип базы данных с её характерными особенностями.

Подсказка: Каждый проект требует своей базы данных.

Ответ:

- **MongoDB** — Документно-ориентированная база данных, хорошо подходит для гибких данных.
- **PostgreSQL** — Реляционная база данных с поддержкой сложных запросов и транзакций.
- **Redis** — База данных для хранения быстрых кэшированных данных в памяти.
- **Нет базы данных** — Проект работает без хранения данных на сервере.

Шаг 04:

Вопрос 3

Сопоставьте вид интерфейса с его назначением.

Подсказка: Разные приложения используют разные интерфейсы.

Ответ:

- **Веб-интерфейс** — Позволяет пользователю взаимодействовать с приложением через браузер.
- **Консольный интерфейс** — Используется через командную строку без графики.
- **API-интерфейс** — Позволяет другим программам взаимодействовать с вашим сервисом через HTTP-запросы.
- **Мобильное приложение** — Интерфейс на смартфоне с сенсорным управлением.

Шаг 05:

Вопрос 4

Сопоставьте тип хостинга с его особенностями.

Подсказка: Выбирайте хостинг в зависимости от проекта.

Ответ:

- **VPS** — Виртуальный сервер с полным доступом к настройкам.

- **Shared Hosting** — Совместный сервер, подходит для небольших сайтов.
- **Cloud Hosting** — Масштабируемый сервер в облаке для высоконагруженных проектов.
- **Локальный сервер** — Проект запускается на компьютере разработчика без внешнего доступа.

🔗 Шаг 06:

Вопрос 5

Сопоставьте метод развертывания с его преимуществами.

Подсказка: Разные проекты требуют разных подходов к развертыванию.

Ответ:

- **Docker** — Контейнеризация приложения для быстрого и повторяемого запуска.
- **GitHub Pages** — Хостинг статических сайтов прямо из репозитория.
- **Heroku** — Платформа для быстрого деплоя веб-приложений с минимальными настройками.
- **Локальный запуск** — Тестирование и запуск приложения только на компьютере разработчика.

🔗 Шаг 07:

Вопрос 6

Сопоставьте технологию Go с её применением.

Подсказка: Go подходит для разных типов проектов.

Ответ:

- **net/http** — Создание веб-серверов и обработки HTTP-запросов.
- **goroutines** — Параллельное выполнение задач для ускорения работы приложения.
- **channels** — Обмен данными между параллельными потоками безопасным способом.
- **templates** — Генерация HTML-страниц для веб-приложений.

🔗 Шаг 08:

Вопрос 7

Сопоставьте тип проекта Go с примером использования.

Подсказка: Каждый пример уникален.

Ответ:

- **Чат-бот** — Автоматический помощник для Discord или Telegram.
- **Веб-магазин** — Интернет-магазин для продажи игрушек или книг.
- **API-сервис** — Сервис для предоставления данных о погоде через HTTP.
- **Скрипт для автоматизации** — Программа для массового переименования файлов на компьютере.

🔗 Шаг 09:

Вопрос 8

Сопоставьте тип тестирования с его назначением.

Подсказка: Разные тесты проверяют разные аспекты проекта.

Ответ:

- **Unit-тест** — Проверка работы отдельной функции или модуля.
- **Integration-тест** — Проверка взаимодействия нескольких модулей между собой.

- **End-to-end тест** — Проверка работы всей системы от начала до конца.
- **Load-тест** — Проверка, как система ведёт себя при высокой нагрузке.

Шаг 10:

Вопрос 9

Сопоставьте способ хранения данных с его назначением.

Подсказка: Каждый способ хранения имеет свои преимущества.

Ответ:

- **Файл CSV** — Хранение таблиц и данных без базы данных.
- **JSON-файл** — Хранение структурированных данных для обмена с другими приложениями.
- **SQL база данных** — Хранение данных с возможностью сложных запросов.
- **NoSQL база данных** — Хранение неструктурированных данных с высокой масштабируемостью.

Шаг 11:

Вопрос 10

Сопоставьте инструмент разработки с его назначением.

Подсказка: Каждый инструмент помогает в своей области.

Ответ:

- **VS Code** — Редактор кода с поддержкой множества языков и расширений.
- **GoLand** — IDE, специализированная для разработки на Go.
- **Postman** — Инструмент для тестирования API-запросов.
- **Docker Desktop** — Запуск и управление контейнерами локально.